





Introduction

- ➔ Architecture d'une application web
- ➔ Les piliers de Angular
- ➔ Créer un premier projet

Application web

Architecture

1. Architecture d'une application web



Une application web est constituée de

- Un *Front-End* (client, navigateur)
- Un *Back-End* (serveur pour les données, les services, etc...)

Parfois on aura en plus

- un serveur d'authentification séparé du back-end
- Réseau de diffusion de contenu (*Content Delivery Network - CDN*)
- du monitoring

2. Qui fait quoi ?

Front (Angular)

UI/UX, état UI (*signals*), validation UI, navigation (*router*), ...

Back (API)

Règles métier globales, validation serveur, sécurité, auth/roles, persistance, intégrations externes.



Exemples

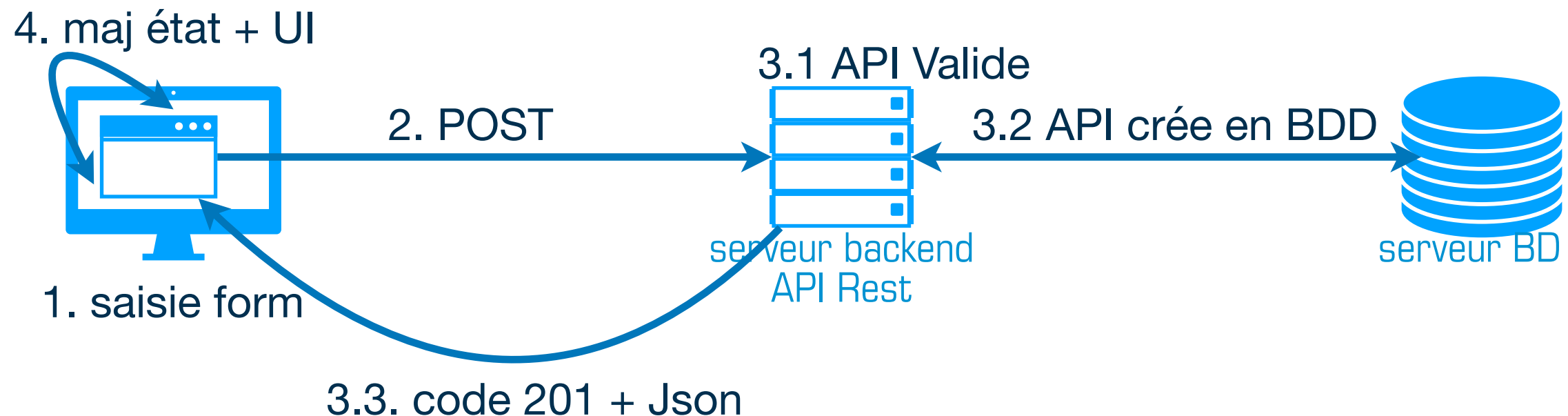
- *Front* : formatage de date, masques de saisie, feedback instantané.
- *Back* : quotas, droit d'accès, cohérence transactionnelle.

Question :

Valider qu'un email est unique : front ou back ?

3. Flux typique d'une fonctionnalité "CRUD"

1. L'utilisateur saisit un formulaire (front).
2. Validation réactive (front) → POST vers API.
3. L'API valide/rejette, crée en BDD, renvoie 201 + ressource.
4. Le front met à jour l'état (*signal*) et l'UI.



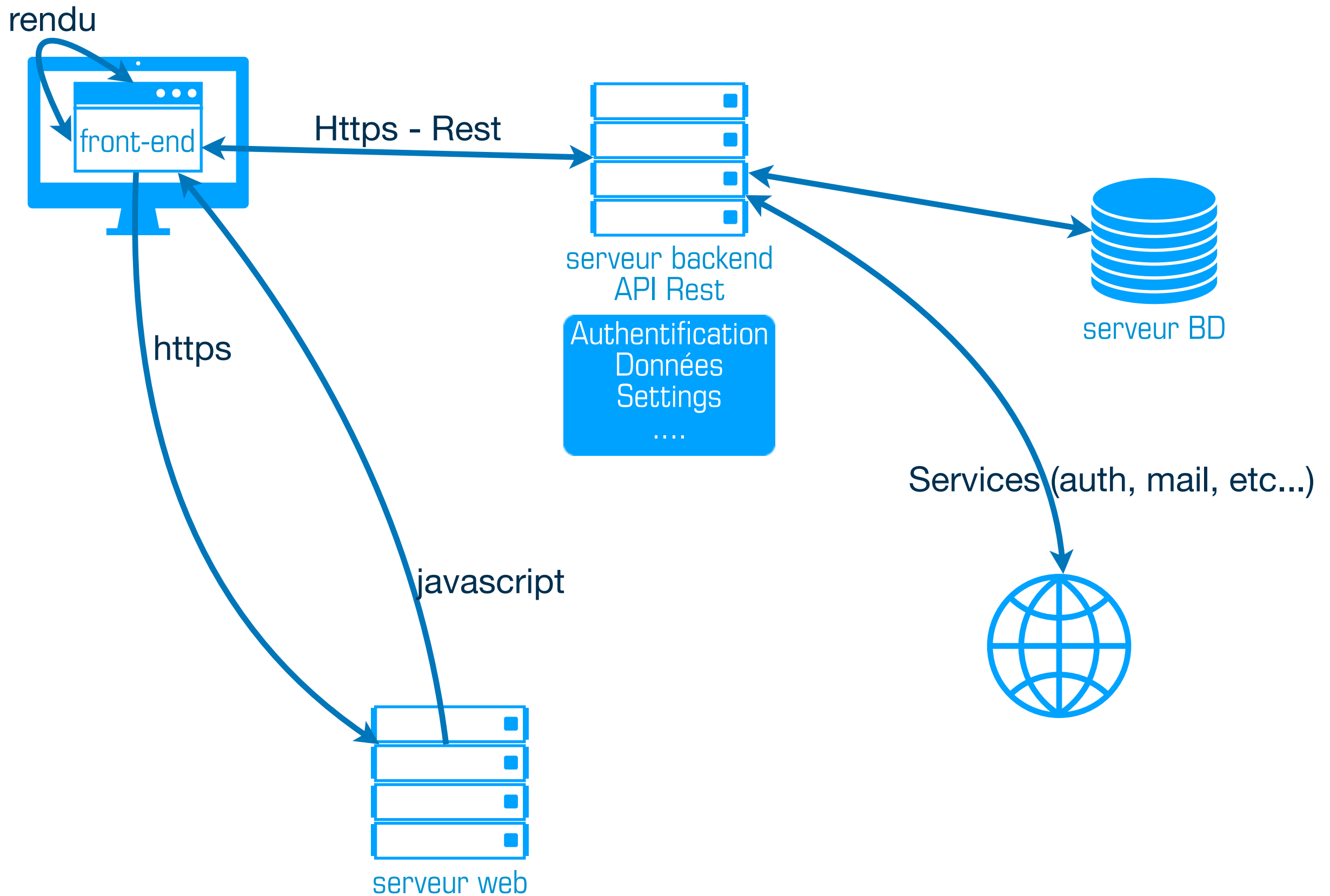
4. PHP vs Angular/React : rôles différents

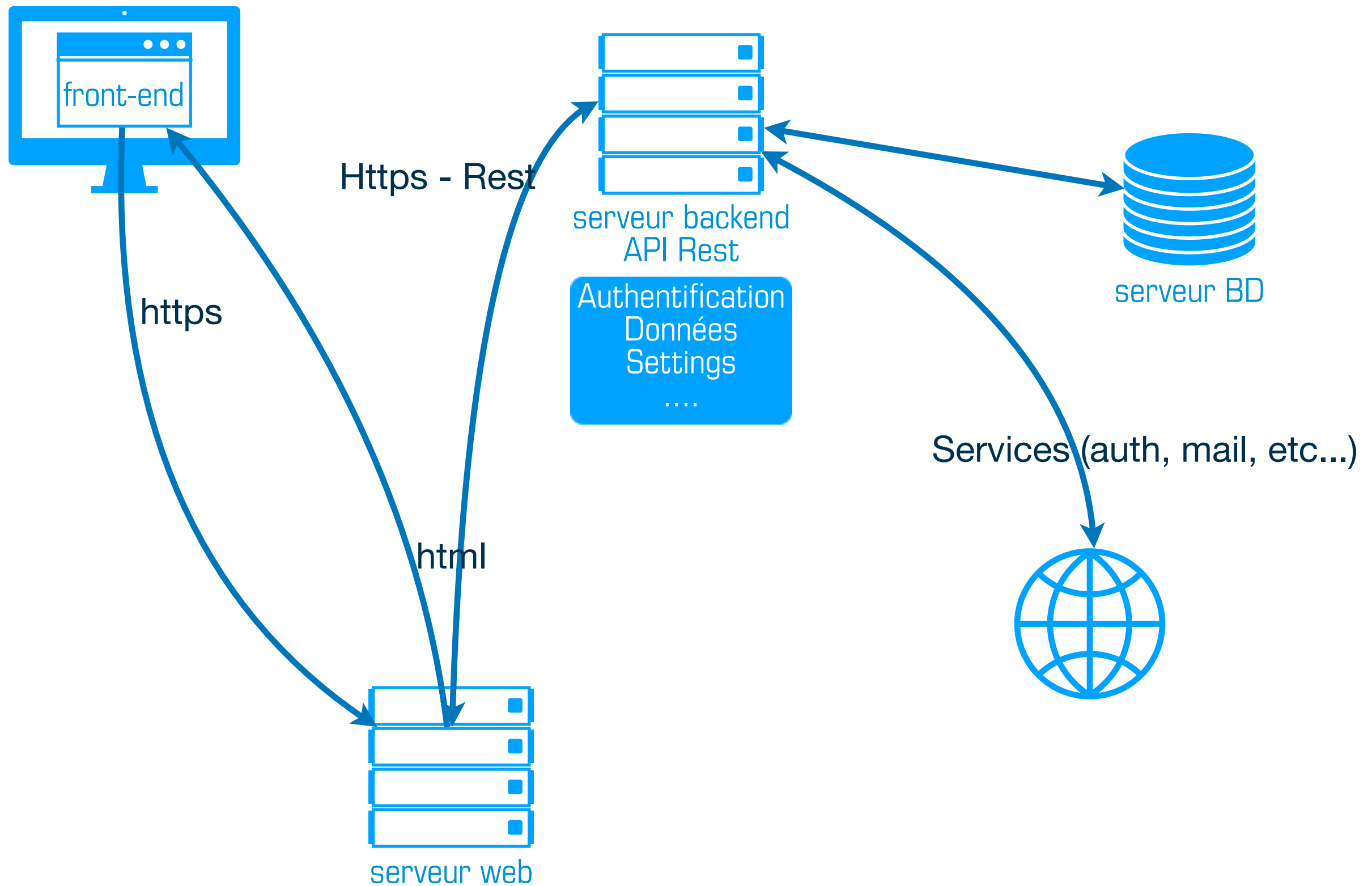


PHP (classique) : rendu côté serveur (SSR), le serveur génère du HTML complet - peu de JS (AJAX)

Angular/React : SPA (Single Page Application) côté client, le navigateur exécute JS, consomme une API.

On peut combiner (SSR/Universal), mais Angular 20 en cours : on se focalise sur SPA + API.





Introduction à Angular

1. Angular, késako ?

Angular est un framework open-source développé par Google pour construire des applications web modernes, modulaires et réactives.

!! Attention : Angular $\neq$ AngularJS (ancienne version, abandonnée).

Caractéristiques principales

- Construit sur TypeScript (*type strict*, POO, tooling riche).
- Basé sur une architecture par *composants* : chaque fonctionnalité est isolée et réutilisable.
- *Réactivité* simple et performante (*signal()*)
- Intègre nativement des bibliothèques officielles pour :
 - *Routage* (navigation entre pages).
 - *Formulaires* réactifs (saisie et validation).
 - *HttpClient* (communication avec une API).
 - *Services & Injection de dépendances* (organisation et testabilité).

2. Les piliers d'Angular 20



Angular 20 s'appuie sur 8 blocs essentiels que nous allons explorer ensemble :

1. *Composants* : brique de base, autonomes, optimisés (**OnPush**).
2. *Communication entre composants* : **input()** et **output()**.
3. *Contrôle de flux* : **@if**, **@for**, **@switch**.
4. *Gestion de l'état* réactif avec les Signals (**signal**, **computed**, **effect**).
5. *Formulaires réactifs* : validations et gestion des erreurs.
6. *Routage* moderne : navigation multi-vues.
7. *Services & Injection* de dépendances : séparation logique métier / UI.
8. *HTTP & backend* : communication RESTful via **HttpClient**.

3. Installation d'Angular

Pour travailler avec Angular, il faut :

- Node.js + npm (gestionnaire de packages).
- Angular CLI : outil officiel pour générer, tester et déployer une app.

➔ Installation de la CLI

```
bash
npm install -g @angular/cli
```

✓ vérification

```
bash
ng version
```

4. Créer un premier projet Angular

Avec Angular CLI

```
bash
ng new my-app --standalone --strict
cd my-app
ng serve
```

Puis aller sur <http://localhost:4200>.

StackBlitz / Angular Playground :

- Alternative en ligne pour tester Angular sans rien installer.
- Utile pour des prototypes rapides ou pour déboguer en TP.

En cours : nous travaillerons principalement avec la CLI en local, mais StackBlitz ou playground peut être utilisé ponctuellement pour faire des tests rapides

```
fiorio@MacBook-Pro-de-Christophe%f:~/ → mkdir Exemples
fiorio@MacBook-Pro-de-Christophe%f:~/ → ng new my-app --standalone

? Which stylesheet format would you like to use? Sass (SCSS) [ https://sass-lang.com/documentation/syntax#scss
? Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? no
CREATE my-app/README.md (1066 bytes)
CREATE my-app/.editorconfig (274 bytes)
CREATE my-app/.gitignore (587 bytes)
CREATE my-app/angular.json (2764 bytes)
CREATE my-app/package.json (1038 bytes)
CREATE my-app/tsconfig.json (1012 bytes)
CREATE my-app/tsconfig.app.json (424 bytes)
CREATE my-app/tsconfig.spec.json (434 bytes)
CREATE my-app/.vscode/extensions.json (130 bytes)
CREATE my-app/.vscode/launch.json (470 bytes)
CREATE my-app/.vscode/tasks.json (938 bytes)
CREATE my-app/src/main.ts (250 bytes)
CREATE my-app/src/index.html (291 bytes)
CREATE my-app/src/styles.scss (80 bytes)
CREATE my-app/src/app/app.component.scss (0 bytes)
CREATE my-app/src/app/app.component.html (19903 bytes)
CREATE my-app/src/app/app.component.spec.ts (916 bytes)
CREATE my-app/src/app/app.component.ts (303 bytes)
CREATE my-app/src/app/app.config.ts (310 bytes)
CREATE my-app/src/app/app.routes.ts (77 bytes)
CREATE my-app/public/favicon.ico (15086 bytes)
Installing packages (npm)...
✓ Packages installed successfully.
  Successfully initialized git.
fiorio@MacBook-Pro-de-Christophe%f:~/ → cd Exemples
fiorio@MacBook-Pro-de-Christophe%f:~/ → cd my-app
fiorio@MacBook-Pro-de-Christophe%f:~/ on main → ng serve

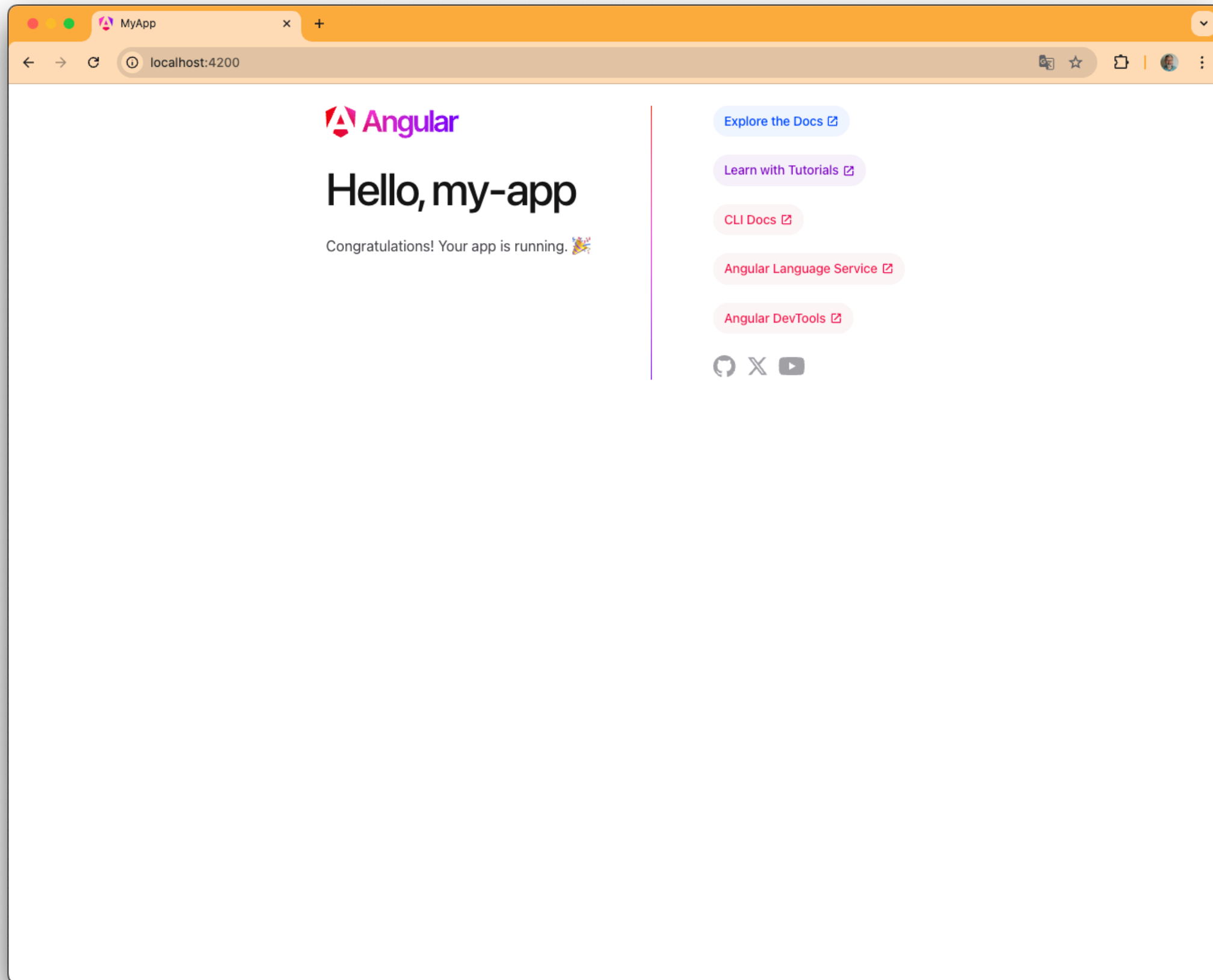
Initial chunk files | Names | Raw size
polyfills.js | polyfills | 90.20 kB |
main.js | main | 22.64 kB |
styles.css | styles | 96 bytes |

| Initial total | 112.93 kB

Application bundle generation complete. [1.569 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  + Local: http://localhost:4200/
  - press h + enter to show help
```

Commandes pour générer l'application et la lancer



Résultat obtenu

5. Structure d'un projet Angular

Un projet Angular généré par la CLI contient :

- `/src/app` → code de l'application.
- `app.component.ts` → composant racine.
- `app.component.html` → template.
- `app.component.css` → styles.
- `main.ts` → point d'entrée de l'application.

👉 Un projet Angular c'est

- un ensemble de composants
- des services
- du routage.



I. Les composants

- ➔ Rôle de la métadonnée `@Component`.
- ➔ Structure d'un composant
- ➔ Construire un composant autonome sans NgModule.

1. Qu'est-ce qu'un composant Angular ?

Un composant = 3 parties :

- Classe TypeScript (logique & état).
- Template HTML (structure visuelle).
- Style CSS (apparence).

👉 Angular relie ces trois éléments via la métadonnée `@Component`.

```
.ts
import { Component, ChangeDetectionStrategy } from '@angular/core';
@Component({
  selector: 'app-hello', //nom de balise html
  template: '<h1>Hello Angular 20!</h1>', //structure du composant
  styles: ['h1 { color: darkblue; }'], //mise en forme
  changeDetection: ChangeDetectionStrategy.OnPush // ↗ performance
})
export class HelloComponent {}
```


2. Un mot sur OnPush

👉 Par défaut (`ChangeDetectionStrategy.Default`), Angular vérifie tout l'arbre de composants à chaque événement :

- un clic,
- une frappe clavier,
- une requête HTTP terminée,
- un `setTimeout`...

Cela garantit que l'UI est toujours synchronisée avec l'état, mais cela peut être coûteux sur une grosse application.

👉 Avec `ChangeDetectionStrategy.OnPush`, on dit à Angular :

“Ne vérifie ce composant que si ses inputs changent ou si un signal observé change.”

Le composant ne sera pas recalculé inutilement et Angular fait confiance au développeur pour signaler les vrais changements.

Mais

- pas de détection sur mutation interne $\implies$ il faut changer la ref.
- pas de détection si un service modifie un signal ou un observable $\implies$ il faut faire appel à `signal()` ou que le composant utilise un `computed()` ou `effect()`

👉 À retenir :

- ▶ `Default`: chaque événement $\rightarrow$ tout l'arbre est vérifié et plus simple à utiliser
- ▶ `OnPush` : chaque événement $\rightarrow$ SEULEMENT les composants dont les inputs/signals ont changé mais discipline plus stricte (e.g. `markForCheck()`)

3. Exercice 1 : Créer un composant HelloWorld



1. Générer un composant avec la CLI :

```
bash
```

```
ng generate component student/hello-student
```

2. Modifier son template pour afficher :

```
html
```

```
<h2>Hello, World! Bienvenue dans l'appli Angular 🥳 </h2>
```

3. Vérifier que le composant s'affiche dans `app.component.html`.

4. Afficher les données dans un template

Un composant peut définir des propriétés dans sa classe.

On peut les afficher dans le template grâce à l'interpolation.

Une expression **exp** entre accolades `{{ exp }}` dans le template sera évaluée et le résultat de l'évaluation sera affiché :

```
.ts
@Component({
  selector: 'app-student',
  template: `<p>Promo : {{ promo }} année {{ 2024+1 }}</p>`
})
export class StudentComponent {
  promo = 'DaMS3'
}
```

Angular remplace `{{ promo }}` par DaMS3 et `{{ 2024+1 }}` par 2025

5. Problème courant : "not a known element"

Si on essaie d'utiliser `<festival-card>` dans un autre composant, par exemple le composant `AppComponent`, Angular peut afficher :

```
NG0303: 'festival-card' is not a known element
```

Pourquoi ?

Parce que chaque composant est désormais par défaut *standalone*

⇒ ils sont indépendants les uns des autres et donc doivent être importés explicitement dans ceux qui les utilisent

```
.ts
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [FestivalCardComponent], // import obligatoire
  template: `<festival-card></festival-card>`
})
export class App {}
```

6. Mini défi : StudentCard

1. Générer un composant `StudentCard`.
2. Ajouter des propriétés :
 - `firstname` = `'Alice'`
 - `name` = `'Demers'`
 - `filiere` = `'DaMS'`
 - `promo` = `3`
3. Afficher ces données dans une carte simple (HTML)

Résultat attendu



7. Mise en forme avec les pipes

Les pipes | permettent de transformer une donnée avant affichage.

```
{{ valeur | pipeName }}
```

Exemple

```
<p>Nom : {{ name | uppercase }}</p>  
<p>Email : {{ email | lowercase }}</p>  
<p>Date d'inscription : {{ createdAt | date:'longDate' }}</p>
```

On peut définir ses propres pipes mais il y en a déjà de prédéfinis.

Les plus utiles :

- uppercase / lowercase
- date (formats variés)
- currency, percent, json

8. Exercice 2: StudentCard améliorée



- 👉 Améliorer l’affichage en utilisant des pipes.
- 1. Rajoutez propriété date d'inscription initialisée avec `new Date()`
- 2. Rajouter le prix de l'inscription
- 3. Afficher le nom de l'étudiant en majuscules.
- 4. Afficher la date d'inscription en format long, par exemple “3 septembre 2025”.
- 5. Afficher le prix de l'inscription avec le symbole monétaire
- 6. Afficher les autres propriétés

Résultat attendu :



Nom : Bob DUGON



inscrit le 3 septembre 2025



payé 628 €



Promo : DaMS 4



Récapitulatif - Composants

👉 Un composant Angular = Classe (TS) + Template (HTML) + Style (CSS)

- `app.component.ts` : définir la logique en TypeScript
- `app.component.html` : décrire la structure en HTML
- `app.component.css` : styliser l'affichage avec CSS/SCSS
- Annotation `@Component`: relie classe, template et style.

10. Compétences acquises

Compétences acquises

- *Identifier* la structure d'un projet Angular générée par la CLI
- *Expliquer* le rôle d'un composant Angular et de la métadonnée `@Component`.
- *Créer* un composant autonome (`standalone`) dans modules (`NgModule`)
- *Afficher* des données avec interpolation et mise en forme (`{{}}` et pipe)
- *Appliquer* `ChangeDetectionStrategy.OnPush` pour optimiser la détection des changements et améliorer les performances.

11. Projet fil rouge (lancement)

👉 On lance le projet final dès maintenant

1. Initialiser un projet Angular :

```
bash
```

```
ng new festival-votrenom-app --standalone  
cd festival-votrenom-app  
ng serve
```

2. Vérifier le bon fonctionnement (lancement via ng serve).

3. Ajouter un premier composant `FestivalCardComponent` avec un festival en dur (pas de prix, ni de date de création pour le projet)

4. Afficher la carte dans le `app.component.html`

👍 On reviendra sur ce projet à chaque séance pour y ajouter de nouvelles fonctionnalités.



II. Communication entre composants

- Comprendre la communication parent $\longleftrightarrow$ enfant
- Utiliser `input()` pour transmettre des données
- Utiliser `output()` pour émettre des événements
- Appliquer ces mécanismes dans notre projet Festival

1. Pourquoi communiquer entre composants ?

Un composant n'est jamais seul : il fait partie d'un système hiérarchique.

Il faut échanger des données :

- Parent → Enfant : le parent transmet des données (**input** ())
- Enfant → Parent : l'enfant émet des événements (**output** ())

Exemple :

- **StudentList** affiche plusieurs **StudentCard**.
- Chaque **StudentCard** (enfant) reçoit les infos d'un étudiant
- On pourrait avoir un bouton dans **StudentCard** pour demander au parent **StudentList** de supprimer cet étudiant

2. Communication parent → enfant (**input()**)

Avec Angular 20 : on utilise **input()** (fonction, pas décorateur) pour permettre à un parent de passer une donnée à un enfant.

Exemple

 composant enfant

student-card.ts

```
import { Component, input } from '@angular/core'
@Component({
  selector: 'student-card',
  template: 'le nom de l'étudiant est {{ firstname() name() }}'
})
export class StudentCardComponent {
  firstname = input<string>(); name = input<string>()
  program = input<string>(); programyear = input<number>()
  price = input<number>(); subscribed = input<Date>()
}
```

!! Notes importantes :

- `input` est un signal, donc si vous voulez utiliser la valeur du signal, il faut appeler la fonction `signal()`.
- on peut appeler un composant sans lui passer le paramètre demandé, le type réel de `input<T>()` est donc `T | null`
- dans le template `{{ param() }}` peut être null et donc `{{ param().propriété }}` provoque une erreur : 3 solutions
 - 1) chainage optionnel: `param()?.propriété` $\Rightarrow$ 🤔 peut laisser un espace vide à l'affichage
 - 2) récupérer la valeur si elle existe, sinon ne pas afficher le composant :

```
@if(param()){  
  @let value = param()!  
  ...  
}
```

- 3) rendre obligatoire le passage du paramètre

```
param = input.required<T>()
```

👉 composant parent

```
student-list.component.html  
  
<student-card  
  firstname = "Alice"    [name] = "'Demers'"  
  [program] = "'DaMS'"  [programyear] = "4"  
  [price]   = "628.0"    [subscribed] = "new Date()" />
```

👉 Alice est transmis comme une chaîne brute (HTML) alors que pour `name` on évalue l'expression entre `''` qui est une chaîne de caractère aussi mais pourrait être le nom d'une variable (sans les `'`) et c'est alors la valeur de la variable qui serait transmise à `name`

3. Passer des variables à un composant

Avec des crochets [], Angular évalue une expression TypeScript et l'applique.

 **Exemple : passer une variable à un composant**

 composant parent

```
.ts
```

```
name = 'Alice'
```

```
.html
```

```
<student-card [firstname]="name"></student-card>
```

 la valeur de `name` est injectée dans la propriété `firstname` du composant enfant

```
.html
```

```
<student-card [firstname]=" 'name' "></festival-card>
```

 ici `name` n'est plus la propriété mais la chaîne de caractère `'name'`. C'est elle qui est injectée dans la propriété `firstname` du composant enfant

Exemple : Lier une propriété HTML native

 composant parent

```
.ts
isDisabled = true
.html
<button [disabled]="isDisabled"></button>
```

disabled est une propriété native HTML :

- si `isDisabled = true` → le bouton est grisé
- si `isDisabled = false` → le bouton est actif

 Les crochets `[ ]` servent à lier n'importe quelle propriété (Angular ou HTML) à une expression TypeScript.

!! Attention à bien faire la différence entre attribut et propriété :

```
<td [colspan]="{{1+1}}"> // ERREUR: colspan est un attribut
<td [colSpan]="{{1+1}}"> // OK: colSpan est la propriété
```

4. Exercice 3: transmettre des données

1. Créez un composant `StudentList`
 2. Dans `StudentList`, déclarez 2 ou 3 étudiants en propriétés TS (pas dans un tableau pour l'instant)
 3. Dans le template de `StudentList`, insérez plusieurs fois manuellement le composant enfant `StudentCard` pour chaque étudiant déclaré
-  Résultat attendu : vous devez voir s'afficher les 2 étudiants
-  Modifier une valeur de nom, et l'affichage doit changer

5. Communication enfant → parent (**output()**)

Avec Angular 20 : on utilise **output()** (fonction, pas décorateur) qui permet à un enfant d'émettre un évènement que pourra écouter le parent

 **Exemple avec un bouton "Supprimer"**

 composant enfant

```
.ts
import { Component, output } from '@angular/core'
@Component({
  selector: 'student-card',
  templateUrl: './student-card.html'
})
export class StudentCardComponent {
  ...
  remove = output<void>()
}
```

👉 composant parent

```
.html
```

```
<student-card (remove)="onDelete()"></student-card>
```

👉 L'enfant notifie le parent d'une action utilisateur qui `onDelete()` du parent

Remarque sur le typage :

`output<void>()` $\Rightarrow$ l'événement ne transmet rien ;

`output<T>()` $\Rightarrow$ l'événement transmet une valeur de type T ;
`emit()` transmet une valeur en paramètre qui est récupérée par un handler qui prend un paramètre du même type que le output.

Exemple avec un bouton "Supprimer" et un id :

composant enfant

```
.ts
@Component({
  selector: 'student-card',
  template: '<button (click)="remove.emit("id")">Supprimer</button>'
})
export class StudentCardComponent {
  remove = output<string>()
}
```

composant parent

```
.html
<student-card (remove)="onDelete($event)"></student-card>
```

 L'enfant notifie le parent d'une action utilisateur qui sera traitée par la fonction `onDelete()` du parent

Remarque : si le `output` est typé, le paramètre `$event` dans l'html du parent est obligatoire.

6. Exercice 4: suppression d'un étudiant

👉 **Objectif** : connecter `StudentCard` et `StudentList`.

1. Dans `StudentCard`, ajoutez un bouton “Supprimer” qui émet `remove`.
2. Dans `StudentList`, n'affichez qu'un seul étudiant et écoutez (`remove`) pour masquer la carte.

🔍 Exemple pour aider :

```
parent (html)
<student-card
  firstname    = "Alice"      [name]          = "'Demers'"
  [program]    = "'DaMS'"     [programyear] = "4"
  [price]      = "628"        [subscribed]  = "newDate()"
  (remove)="..."
  [hidden]="hideStudent">
</student-card>
```



Récapitulatif - Communication entre composants

- Données du parent vers l'enfant via `input()`
- Événements de l'enfant vers le parent via `output()`
- Import explicite des composants enfants (`standalone`)
- Bindings :
 - passage de paramètres (input) : `[]`
 - réception d'une valeur (output) : `()`
- Différence de binding pour les input:
 - sans crochet `[]` : valeur brute (chaîne HTML).
 - avec crochet `[]` : expression TypeScript évaluée

8. Compétences acquises

Compétences acquises

- *Importer* un composant enfant dans un composant parent.
- *Utiliser* `input<T>()` pour transmettre des données typées du parent vers l'enfant.
- *Différencier* une valeur brute HTML d'une expression TS dans un binding.
- *Utiliser* `output<T>()` pour émettre un événement.
- *Implémenter* une interaction enfant $\longrightarrow$ parent avec `.emit()`, avec ou sans valeur (`output<void>()` ou `output<string>()`).

9 Projet fil rouge : communication enfant $\longleftrightarrow$ parent



Objectif : illustrer comment un composant enfant peut envoyer un événement au parent grâce à `output()` tout en mettant en œuvre une communication parent $\longleftrightarrow$ enfant

1. Créez un composant `FestivalCard` qui affiche des 3 propriétés `name`, `location`, `year` et un bouton supprimer
 2. Créez un composant `FestivalList` qui :
 - a. crée 3 festivals en dur (`festival1`, `festival2`, `festival3`)
 - b. affiche 3 cartes `FestivalCard`, un pour chacun des festivals
 3. Faites en sorte que lorsqu'on clique sur "Supprimer" d'une carte, le parent masque la carte correspondante
- 💡 Astuce : utilisez des propriétés booléennes (`hide1`, `hide2`, `hide3`)

Résultat attendu :

1. au départ : trois cartes affichées ;
2. après clic sur “Supprimer” dans une carte, elle disparaît

Bonus pour les rapides 🚀

- Améliorez `FestivalList` pour gérer les suppressions avec un seul handler.
- Idée : l’enfant peut émettre une donnée utile (par ex. l’année du festival (un seul festival par an).
- Le parent utilise cette donnée pour déterminer quelle carte masquer.

Résultat attendu :

3. avec le bonus → même comportement mais un seul handler dans le parent qui enlève la bonne Card quelle que soit celle qui a été demandée à être supprimer.

Bonus pour les plus rapides 🚄

- Refactorisez `FestivalList` pour gérer les festivals dans un tableau

🖥️ Résultat attendu

Même résultat que précédemment mais un tableau qui permettra à l'avenir d'ajouter plus facilement des festivals mais aussi d'utiliser les flux de contrôle



III. Structures de contrôle et styles dynamiques

- ➔ Utiliser `@if`, `@for`, `@switch` pour contrôler l’affichage.
- ➔ Appliquer des classes et styles dynamiques avec `[class]` et `[style]`.
- ➔ Réécrire du code “legacy” (`*ngIf`, `ngClass`) en syntaxe moderne.
- ➔ Intégrer ces concepts dans les composants et dans le projet fil rouge.

1. Intermède Typescript : Interface vs Class

Interface

- sert à décrire la forme des données ;
- purement statique (compilation TS) : n'existe pas au runtime $\Rightarrow$ aucun impact à l'exécution.

Idéal pour :

- typer des données simples (ex. issues d'une API REST) ;
- assurer la cohérence du code.

Exemple :

```
.bash
```

```
ng g interface student/student-dto
```

```
.ts
```

```
export interface StudentDTO {  
  id?      : number // pas défini à la création d'un student  $\Rightarrow$  défini par l'API REST  
  firstname : string  
  name      : string  
  program   : string  
  programYear : number  
  price     : number  
  subscribed : Date  
}
```

Class

- sert à décrire des données + logique métier.
- existe au runtime (après transcompilation).
- peut avoir un constructeur, des méthodes, des getters.

Idéale pour :

- modèles métiers avec comportements ;

 **Exemple :**

```
.bash
ng g class student/student --skip-tests
```

```
.ts
export class Student {
  constructor(
    public readonly id: number | null,
    public firstname: string,
    ...
  ) { }
  get fullname(): string {
    return `${this.firstname} en ${this.name}`
  }
  get isFinalYear(): boolean {
    return this.promoYear === 5
  }
}
```

Qu'utiliser, Class ou Interface ?

Interfaces vs Classes en Angular :

- *Interfaces* → données passives (*typage*)
- *Classes* → données actives (*logique métier*)

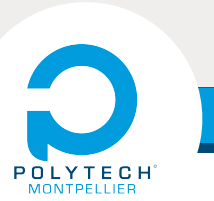


À retenir

Dans Angular, on utilise surtout des interfaces pour typer les données (par ex. réponses HTTP).

Les classes sont utiles quand on veut enrichir le modèle avec de la logique métier.

2. Pourquoi des structures de contrôle ?



Les données sont dynamiques :

- Un festival peut être en cours ou non.
- La liste de festivals peut contenir plusieurs entrées.
- On veut afficher des informations différentes selon la situation.

On a tout le temps besoin de gérer un affichage en fonction des valeurs de certaines données.

⇒ Il est nécessaire de disposer de structure de contrôle permettant cela dans la structure (.html)

Angular fournit une syntaxe moderne, lisible et performante.

3. Syntaxe moderne contre syntaxe classique



Cette syntaxe a été revue depuis Angular 17+ (expérimentale) et stabilisée depuis Angular 18

L'ancienne syntaxe à base de directives (**ngIf*, **ngFor*, *ngClass*, *ngStyle*) était parfois lourde et peu lisible.

Nouvelle syntaxe (Angular 17+) :

- plus proche de TypeScript ;
- plus performante ;
- plus naturelle et donc plus simple à utiliser.

4. Structure de contrôle if-then-else : @if

Avant (legacy) :

```
.html
<div *ngIf="student.isFinalYear; else notLastYear">
  <p>🍷 Élève {{ student.name }} en dernière année</p>
</div>
<ng-template #notLastYear>
  <p>🚶 Élève {{ student.name }} a encore du chemin à faire</p>
</ng-template>
```

Maintenant (version moderne) :

```
.html
@if (student.isFinalYear) {
  <p>🍷 Élève {{ student.name }} en dernière année</p>
} @else {
  <p>🚶 Élève {{ student.name }} a encore du chemin à faire</p>
}
```

5. Exercice 5 : @if avec Student



1. Utilisez maintenant des types pour Student
2. Dans StudentCard afficher une première ligne qui indique si
 - a. "alumni", si c'est un ou une élève inscrite avant 2023
 - b. "étudiant", si c'est un ou une élève inscrite en 2024 ou 2023
 - c. "nouvel inscrit", si c'est un ou une future élève inscrite après 2025

6. Structure de contrôle for : @for

Avant (legacy) :

```
.html
<ul>
  <li *ngFor="let student of student">{{ student.name }}</li>
</ul>
```

Maintenant (version moderne) :

```
.html
<ul>
  @for (student of students; track student.name) {
    <li>{{ student.name }}</li>
  }
</ul>
```

`track` indique quelle propriété *unique* identifie chaque élément.

Pourquoi est-ce important ?

- Angular doit savoir reconnaître chaque élément quand la liste change.
- Avec `track`, Angular réutilise les éléments inchangés et ne met à jour que ce qui change.

7. Exercice 6 : afficher une liste de `Students` : `@for`

1. Définissez une interface `Student` et ajoutez-y un `id`
2. Dans `StudentList`, définissez un tableau `students: Student[]` avec au moins 3 `Students`.
3. Utilisez `@for` pour parcourir le tableau et afficher chaque étudiant via `<student-card>`.
4. Utilisez `track` avec `.id` pour assurer l'efficacité du rendu.

 Résultat attendu :

- la liste affiche les 3 `Student` dans 3 cartes ;
- le `track` permet à Angular de réutiliser les cartes existantes si la liste est modifiée (ajout/suppression plus tard).

8. Structure de contrôle switch-case : @switch

Avant (legacy) :

.html

```
<div [ngSwitch]="student.location">
  <p *ngSwitchCase="'Montpellier'">☀ Sud de la France</p>
  <p *ngSwitchCase="'Toulouse'">☁ Sud-ouest de la France</p>
  <p *ngSwitchDefault>☁ Autre région</p>
</div>
```

Maintenant (version moderne) :

.html

```
@switch (student.location) {
  @case ('Montpellier') { <p>☀ Sud de la France</p> }
  @case ('Toulouse')    { <p>☁ Sud-ouest de la France</p> }
  @default { <p>☁ Autre région</p> }
}
```

9. Exercice 7: afficher une liste d'étudiants avec @switch



Reprenez l'exercice avec le @if mais utilisez cette fois @switch

 Résultat attendu : même affichage que dans l'exercice précédent

10. Styles dynamiques

Agir sur la class de l'élément html

avant :

```
<p [ngClass]="{ highlight: isImportant }">Texte</p>
```

maintenant :

```
<p [class.highlight]="isImportant">Texte</p>
```

Agir sur le style de l'élément html

avant :

```
<p [ngStyle]="{ color: isRed ? 'red' : 'black' }">Texte</p>
```

maintenant :

```
<p [style.color]="isRed ? 'red' : 'black'">Texte</p>
```


Nouvelle Syntaxe

- plus simple, plus lisible
- liaison directe avec la propriété du DOM
- plus cohérent avec le DOM, plus performant : ces bindings sont directement compris par le navigateur, contrairement à `ngClass` et `ngStyle` qui faisaient une étape de transformation intermédiaire.

11. Exercice 8 : classes et styles

👉 **Objectif** : mettre en valeur les étudiantes et étudiants de dernière année.

- Créez une classe CSS `.highlight` avec un style distinctif qui permet de mettre en valeur un affichage
- Dans `StudentCard`, appliquez-la uniquement si `isFinalYear`.
- Vérifiez que seul les étudiants et étudiantes en dernière année sont mises en avant.



Récapitulatif : structures de contrôle et styles dynamiques

👉 Les structures de contrôle Angular 20 permettent d'adapter dynamiquement l'affichage :

- `@if ... @else` : affichage conditionnel
- `@for (...; track ...)` : boucle sur une liste avec identifiant unique
- `@switch` : choix multiple selon valeur

👉 Les styles dynamiques permettent d'adapter l'apparence :

- `[class.nom]` : appliquer une classe conditionnellement.
- `[style.prop]` : modifier un style en fonction d'une expression.

13. Compétences acquises

Compétences acquises

- *Utiliser* `@if`, `@for`, `@switch` pour contrôler et adapter l’affichage.
- *Comprendre* l’importance de `track` dans `@for` pour améliorer la performance.
- *Appliquer* des classes et styles dynamiques avec `[class]` et `[style]`.

14. Projet fil rouge

👉 On enrichit notre app Festival :

1. Festival doit maintenant gérer une liste de festivals (au moins 3)
2. Chaque FestivalCard affiche :
 - nom, localisation, année, état (@if).
 - région (@switch).
3. Mettre en valeur le festival en cours :
 - a. La carte comporte une bordure colorée
 - b. La carte possède un bandeau dans le style `.highlight` et qui précise : Événement principal : *'nom du festival' 'année du festival'*
4. Bonus 🌩️ : afficher un compteur de festivals qui indique le nombre de festival dans la liste



IV. Gestion de l'état avec les Signals

- Comprendre le concept de signal réactif.
- Utiliser `signal()` pour gérer un état local.
- Utiliser `computed()` pour dériver un état.
- Appliquer les signals à des exemples concrets.

1. Pourquoi les Signals ?

Les applications ont besoin de gérer un état (données qui évoluent).

Avant Angular 17, il y avait deux solutions :

- variables classiques, mais pas complètement réactives.
- RxJS `BehaviorSubject`, puissant, mais complexe pour les cas simples.

```
Avant : BehaviorSubject<number>(0)
count$ = new BehaviorSubject(0);
count$.next(1);
count$.subscribe(v => console.log(v));
```

```
Maintenant → signal(0)
count = signal(0);
count.set(1);
console.log(count());
```

Angular 18+ : les Signals deviennent l'API officielle de gestion de l'état.

⇒ Réactivité simple, performante et intuitive.

2. `signal()`

Définition :

Un signal c'est une boîte qui contient une valeur.

- Quand la valeur change, Angular prévient automatiquement les parties du code qui l'utilisent.
- Tu peux y mettre tout type de donnée (un nombre, un texte, un tableau, un objet...).
- Pour lire ce qu'il y a dans la boîte, tu fais `signal()`.
- Certains signaux peuvent être changés (`writable`), d'autres seulement consultés (`read-only`).

💬 En clair, un signal

- crée une valeur réactive ;
- est accessible comme une fonction (ex: `count()`).
- est modifiable avec `.set()` ou `.update()` lorsqu'il est writable

```
.ts
import { WritableSignal , signal } from '@angular/core'

count : WritableSignal<number> = signal<number>(0)
// lecture
console.log(count()) // 0
// écriture
count.set(1) // 1
count.update(c => c+1) // 2
```

👉 Angular mettra à jour automatiquement le template lié à ce signal.

3. Exercice 9 : compteur réactif

Créez un CounterComponent qui :

1. contient un signal `count = signal(0)`.
2. Affichez sa valeur dans le template
3. contient deux boutons :
 - “+1” $\implies$ `count.update(c => c + 1)`
 - “Reset” $\implies$ `count.set(0)`

Résultat attendu

À chaque appui sur un des boutons, l'affichage est mis à jour

Bonus pour rapides 🌩️

1. Ajoutez un bouton “+5” qui incrémente le compteur de 5.
2. Ajoutez un bouton “-1” qui décrémente le compteur.
3. Affichez un message conditionnel :
 - “Compteur positif 👍” si `count()` > 0 .
 - “Compteur nul ou négatif 👎” sinon.

4. computed()

- Permet de créer une valeur dérivée à partir d'un signal.
- Recalculée automatiquement si le signal change.

⇒ propriété calculée réactive

```
.ts
count    = signal(2)
dcount    = computed(() => count() * 2)
console.log(dcount())// 4
count.set(3)
console.log(dcount())// 6
```

👉 Pas besoin de gérer les dépendances entre signals manuellement, Angular le fait.

🔍 Exemple plus complexe :

```
lastYearStudentsCount = computed(() =>
  this.students().filter(f => f.isFinalYear).length
)
```

5. Exercice 10 : compteur avec valeur dérivée

Ajoutez à `CounterComponent` :

1. Un `computed()` qui calcule le double de `count`.
2. Affichez



```
Compteur : X  
Double : 2X
```

Bonus pour les rapides 🚀

1. Créez un second `computed()` qui calcule le triple du compteur.
2. Affichez :



```
Compteur : X  
Double : 2X  
Triple : 3X
```

6. Exercice 11 : Mini-défi

Implémentez une liste réactive avec (vous pouvez compléter votre `ListStudent`):

1. Un signal `items` d'une liste de `string`.
2. Un champ texte + bouton "Ajouter" (pour l'instant on utilisera des inputs HTML classiques)
3. Le template affiche la liste avec `@for`.
4. Ajoutez un `computed()` pour calculer le nombre d'éléments.
5. Affichez avant la liste :



Vous avez X éléments dans la liste.

💡 Pour récupérer la valeur du champ texte, vous pouvez utiliser un input html avec un template référence variable.

.html

```
<input #nameInput type="text" placeholder="Nom">
<button (click)="addName(nameInput.value)">
  Ajouter
</button>
```

Bonus pour rapides 🌩️

1. Ajoutez un bouton “Vider la liste” qui réinitialise le signal.
2. Ajoutez une logique :
 - si la liste est vide alors affichez “Liste vide $\emptyset$ ”.
 - sinon affichez “Vous avez X éléments.”
3. Faites en sorte que la liste affiche un index numéroté :



```
1 - item..  
2 - item..  
...  
...
```

4. Faites un composant de votre "formulaire" de saisie
5. Faites en sorte qu'il ne s'affiche qu'à l'appuie sur un bouton add et qu'il disparaisse une fois la saisie validée et que l'élément saisi ait été rajouté dans la liste



Recap : Signals

👉 Les Signals en Angular 20

- `signal()` → état local réactif.
- `.set()` / `.update()` → modifier l'état.
- `computed()` → état dérivé, recalculé automatiquement.

8. Compétences acquises

Compétences acquises

- *Créer* et manipuler un signal avec `signal()`.
- *Mettre* à jour un état via `.set()` et `.update()`.
- *Définir* un état dérivé avec `computed()`.
- *Appliquer* la réactivité des signals dans un composant concret.
- *Refactoriser* un projet pour remplacer des propriétés classiques par des `signal()`.
- *Relier* l'état réactif du composant à un affichage concret

9. Projet fil rouge

- 👉 Festivals réactifs : s'adapter à l'évolution du nombre de festivals dans la liste
1. Dans `FestivalListComponent`, remplacer la propriété `festivals`:
`Festival[]` par un signal
 2. Dans le template, afficher la liste avec `@for (f of festivals(); track f.id)`.
 3. Afficher au-dessus : "vous avez x festivals enregistrés"
 4. Jusqu'ici on cachait un festival (hidden). Avec les signals, on peut vraiment mettre à jour l'état et supprimer un festival de la liste : supprimer le festival concerné de la liste quand on appuie sur le bouton supprimer de la carte

Résultat attendu :

La liste doit s'afficher et l'affichage doit s'adapter lorsqu'on supprime un festival : le festival concerné ne doit plus être affiché et le nombre de festival doit avoir été mis à jour.

10. Projet fil rouge :défi

👉 Ajouter dynamiquement des festivals

1. Ajouter une méthode `addFestival(name: string, location: string, year: number)` à `FestivalList`
2. Enrichir `FestivalList` avec un petit formulaire (avec de simples input html):
 - Champ texte pour le nom.
 - Champ texte pour la localisation.
 - Champ nombre pour l'année.
 - Bouton "Ajouter festival".
3. Gérer l'appui sur le bouton "Ajouter festival" afin d'ajouter le festival saisi

🖥️ **Résultat attendu** : la liste s'allonge, le compteur change

Bonus 🌬️ : ajouter un bouton "Vider la liste" qui réinitialise la liste à vide.



V. Gestion de l'état avec les Signals : `effect()`

- ➔ Comprendre le rôle de `effect()` pour réagir aux changements d'un signal.
- ➔ Différencier `signal()` (état), `computed()` (état dérivé), `effect()` (réaction).
- ➔ Savoir comparer Signals et RxJS.
- ➔ Appliquer `effect()`

1. Pourquoi `effect()` ?

`signal()` stocke un état réactif.

`computed()` calcule un état dérivé automatiquement.

Mais parfois il faut agir sur un effet de bord (`console.log`, appel API, `localStorage`, etc.).

 `effect()` permet de déclencher une action lors du changement de valeur d'un `signal()`.

 Exemple simple avec `effect()` :

```
.ts
import { signal, effect } from '@angular/core';

count = signal(0);
constructor() {
  effect(() => {
    console.log('Le compteur vaut :', this.count());
  });
}
```

2. Exercice 12: log d'un état

👉 **Objectif** : observer les changements automatiquement.

1. Créez un signal `username = signal('Alice')`.
2. Ajoutez un `effect()` qui affiche dans la console :



```
Utilisateur courant : Alice
```

3. Ajoutez un bouton “Changer de nom” qui change le nom en `'Bob'`.
4. Vérifiez que le log change sans avoir besoin d’abonnement manuel.

2. Comparaison Signals vs RxJS

RxJS BehaviorSubject (ancien monde) :

```
.ts  
count$ = new BehaviorSubject(0);  
count$.subscribe(c => console.log(c));  
count$.next(1);
```

Signals (nouveau monde) :

```
.ts  
count = signal(0);  
effect(() => console.log(count()));  
count.set(1);
```

👉 Plus simple, typé, sans abonnement/désabonnement.

mais RxJS reste utile pour les flux asynchrones complexes (WebSocket, streams multiples).

Mini défi Student : log des suppressions

👉 Objectif : utiliser `effect()`.

1. Dans `StudentList`, ajoutez un `effect()` qui loggue :

Nombre d'étudiants et étudiantes : X

chaque fois que la liste change.

2. Bonus 🌪️ : logguez aussi le dernier `Student` supprimé.

- 💡 astuce : avant de faire `update()`, stockez le `Student` supprimé dans un `signal<Student | null>`.
- `effect()` loggue alors :

Étudiant ou étudiantes supprimée : Bob Dugon

3. Récapitulatif et compétences acquises

👉 Les trois piliers des Signals

- `signal()` : état local.
- `computed()` : état dérivé.
- `effect()` : réagir aux changements.

🔑 Compétences acquises

- *Créer* un `effect()` pour surveiller un signal.
- *Comparer* `effect()` avec RxJS `.subscribe()`.
- *Appliquer* `effect()` dans un composant Festival.
- *Mettre* en œuvre une logique réactive de suivi.

4. Projet fil rouge : suivi enrichi

👉 On enrichit notre app Festival :

1. Ajouter un `computed()` `activeFestivals` (nombre de festivals où `isCurrent == true`).
2. Ajouter un `effect()` qui loggue :
 - Le nombre total de festivals.
 - Le nombre de festivals actifs.
3. Bonus 🌀 : stocker la liste des festivals en `localStorage` à chaque modification via `effect()`.

5. Problème à résoudre

Limites des `signal()`

- Chaque composant gère son propre `signal()` → propriété locale
- Si plusieurs composants ont besoin du même état (ex. utilisateur, festival courant) :
 - Passage par `Input()` / `Output()`
 - Duplication du code.
 - Risque d'incohérences.
 - Pas de “source de vérité” unique.

 **Exemple** : Tous les composants peuvent avoir besoin de connaître l'utilisateur connecté, etc...

6. Vers une solution

👉 Besoin d'un état partagé et centralisé

- Un signal global injecté comme un service (*notion expliquée dans un prochain cours*).
- Accessible dans plusieurs composants pour une source unique de vérité.

Permet :

- Lecture (`signal()`)
- Écriture via méthodes (`setXxx()`)
- Valeurs dérivées (`computed()`)
- Réactions globales (`effect()`)

7. Introduction Signal Store



NgRx Signal Store

Outil officiel (Angular+NgRx) qui permet de structurer un état partagé :

- `withState` : définir l'état (le signal en lui même).
- `withComputed` : créer des dérivés de l'état
- `withMethods` : ajouter à l'API du store des méthodes permettant d'encapsuler les mutations.

? Question : Quels états dans notre application Fil Rouge devraient être partagés globalement ?



VI. Services & Injection de dépendances

- Expliquer le rôle des services dans Angular.
- Créer un service Angular pour encapsuler une logique métier.
- Utiliser `inject()` pour consommer un service dans un composant.
- Justifier la séparation logique métier (service) / UI (composant).
- Mettre en œuvre un service in-memory comme “faux backend” pour tester vos composants.

1. Concepts clés

Qu'est-ce qu'un **service** Angular ?

- Classe TypeScript, fournie via le système d'injection de dépendances (DI)
- Encapsule une logique métier, des données, ou des appels réseau qui sont *partagés* par plusieurs composants

Injection de dépendances (DI)

- Angular gère le cycle de vie des services
- On évite le `new` → on demande une instance avec `inject(Service)`
- *Services = singleton* par défaut (dans leur scope)

👍 Bonne pratique :

- Les composants se concentrent sur l'UI.
- Les services avec les classes métier portent la logique métier et la communication avec le backend.

2. Exemple : service basique

```
student.service.ts

import { Injectable, signal } from '@angular/core'
import { StudentDto } from '../types/student-dto'

@Injectable({ providedIn: 'root' })
export class StudentListService {
  private readonly _students = signal<StudentDto[]>([
    { id: 1, firstname: 'Alice', name: 'Smith', program: 'DaMS', programYear: 2, price:
1000, subscribed: new Date('2022-09-01') },
    { id: 2, firstname: 'Bob', name: 'Johnson', program: 'DaMS', programYear: 3, price:
1200, subscribed: new Date('2021-09-01') },
  ])
  readonly students = this._students.asReadonly() // Contrat public : lecture seule

  add(student: StudentDto): void { this._students.update(list => [...list, student]) }
  remove(id: number): void {this._students.update(list => list.filter(s => s.id !== id)) }
  removeAll(): void { this._students.set([]) }

  update(partial: Partial<StudentDto> & { id: number }): void {
    this._students.update(list =>
      list.map(s => (s.id === partial.id ? { ...s, ...partial } : s))
    )
  }

  findById(id: number): StudentDto | undefined {return this._students().find(s => s.id===id)}
}
```


2.1 findById() : explications

La méthode `findById()` est définie ainsi :

```
.ts  
findById(id: number): Student | undefined {  
    return this._students().find(s => s.id === id)  
}
```

Pourquoi le retour `Student | undefined` ?

Parce que `.find()` retourne l'élément ou `undefined` si aucun étudiant n'est trouvé $\implies$ TypeScript strict oblige à prendre en compte ce cas, ce qui évite des erreurs à l'exécution.

Exemple d'utilisation dans un composant :

.ts

```
const s = this.svc.findById(123)
```

.html

```
@if (s) {  
  <p>{{ s.name }}</p>  
} @else {  
  <em>Étudiant introuvable</em>  
}
```



À retenir :

Toujours typer explicitement les retours avec `| undefined` si le résultat peut ne pas exister pour forcer à traiter correctement le cas “non trouvé” → code plus robuste.

2.2. Explication de `Partial<T>`

En TypeScript, `Partial<T>` est un utilitaire fourni par le langage. Il prend un type générique `T` et rend toutes ses propriétés optionnelles.

Exemple simple

```
interface Student {  
  id: number  
  name: string  
  year: number  
  email: string  
}  
type StudentPartial = Partial<Student>
```

`StudentPartial` équivaut alors à :

```
.ts  
{  
  id?: number  
  name?: string  
  year?: number  
  email?: string  
}
```

Pourquoi c'est utile dans notre service ?

Dans le `StudentListService`, on a une méthode `update` :

```
.ts
update(partial: Partial<Student> & { id: number }): void {
  this._students.update(list =>
    list.map(s => (s.id === partial.id ? { ...s, ...partial } :
s))
  )
}
```

On exige que l'objet passé contienne au moins un `id` (`{id: number}`), pour savoir quel étudiant modifier.

Plutôt que de devoir fournir tout l'objet `Student` complet à chaque fois, on rend optionnelles les autres propriétés (`name`, `year`, `email`) grâce à `Partial<Student>`.

Ça permet d'écrire :

```
.ts
```

```
// met à jour uniquement l'année  
service.update({ id: 1, year: 3 })  
// met à jour uniquement le nom  
service.update({ id: 2, name: 'Alice Dupont' })
```



Résumé

- `Partial<T>` rend toutes les propriétés de `T` optionnelles.
- On peut le combiner avec la description d'un type pour être sûr d'avoir toujours certaines propriétés (ex ici : { `id`: `number` })
- C'est une pratique courante pour implémenter des méthodes `update` dans des services CRUD.

2.3. Panorama des utilitaires TypeScript

Quelques utilitaires courants :

- `Partial<T>` : Rend toutes les propriétés optionnelles.
- `Readonly<T>` : Rend toutes les propriétés en lecture seule.
- `Pick<T, K>` : Construit un type en ne gardant que certaines propriétés.
- `Omit<T, K>` : Construit un type en excluant certaines propriétés.

Exemples :

```
.ts
type StudentPreview = Pick<Student, 'id' | 'name'>;
// { id: number; name: string }

type StudentNoEmail = Omit<Student, 'email'>;
// { id: number; name: string; year: number }
```

3. Bonnes pratiques — Services & état partagé

Exposer en lecture seule

- *Encapsulation* : seules les méthodes du service modifient l'état (set/update).
- *Contrat clair* : les composants lisent, le service écrit.

!! Collections en mémoire

Problèmes possibles :

- Synchro backend : liste locale rapidement périmée si le serveur change.
- Coût mémoire : grandes collections = conso mémoire + recalculs DOM.
- Granularité : mettre à jour un item déclenche parfois un re-render de toute la liste.

Stratégies recommandées :

- API ciblée : findById(id), search(query), pagination ou lazy loading.
- Cache + invalidation : TTL/"Refresh"/revalidation silencieuse après mutation.
- Stores scopés : état par route/domaine (ex. /festivals/:id) plutôt que global.
- En production, préférez accès ciblés (par id, pagination) + cache avec invalidation plutôt qu'une liste globale en mémoire.

Règles simples

- Exposer des read-only signals (`asReadOnly()`).
- Mutations via méthodes du service (API métier).
- Préférer accès ciblé/paginations aux grosses listes globales.

À retenir

- Composants = UI · Services = logique & données
- Protéger l'état, limiter la taille en mémoire, et penser synchro avec le backend dès la conception.

4. Exemple : consommation dans un composant

```
// student-list.component.ts
import { Component, ChangeDetectionStrategy, inject } from '@angular/core'
import { StudentService } from '../services/student.service'
@Component({
  selector: 'student-list',
  templateUrl: 'student-list.component.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
  host: { class: 'block' }
})
export class StudentListComponent {
  readonly svc = inject(StudentService)
  add(name: string, year: number, email: string): void {
    if (!name || !Number.isFinite(year) || !email) return
    this.svc.add({ id: Date.now(), name, year, email })
  }
  promote(id: number): void {
    const s = this.svc.findById(id)
    if (s) this.svc.update({ id, year: s.year + 1 })
  }
  remove(id: number): void {
    this.svc.remove(id)
  }
}
```

```
// student-list.component.html
<h2>Liste des étudiants ({{ svc.students().length }})</h2>

<!-- Entrées HTML simples (pas encore de formulaires réactifs) -->
<div style="margin:.5rem 0;">
  <input #name type="text" placeholder="Nom">
  <input #year type="number" placeholder="Année">
  <input #email type="email" placeholder="Email">
  <button (click)="add(name.value, year.valueAsNumber,
email.value)">Ajouter</button>
</div>

<ul>
  @for (s of svc.students(); track s.id) {
    <li style="margin:.25rem 0;">
      <strong>{{ s.name }}</strong> – année {{ s.year }} –
      {{ s.email }}
      <button (click)="promote(s.id)">+1 année</button>
      <button (click)="remove(s.id)">Supprimer</button>
    </li>
  }
</ul>
```

5. Exercice 13 : StudentListService

Contexte :

Le `StudentListService` in-memory déjà écrit, et présenté en cours, qui expose :

- un état privé `_students` (signal),
- un signal public en lecture seule `students = _students.asReadOnly()`,
- une méthode `findById(id: number): Student | undefined`.

👉 **Objectif :** Disposer d'un composant `StudentDetailComponent` qui consomme ce service pour afficher le détail d'un étudiant à partir d'un identifiant fourni par le parent.

1. Créer le composant `StudentDetailComponent` (standalone, `ChangeDetectionStrategy.OnPush`) qui prend en input un `id`.
2. Exposez une propriété de lecture `student: Student | undefined` qui est un getter en fonction de l'`id`
3. Faites en sorte que le composant affiche
 - le détail de l'étudiants (name, year, email, ..) si il existe
 - « Étudiant introuvable » (en style highlight) si il n'existe pas
4. ...

4. Intégrez le composant dans un parent, et affichez :

- `<student-detail [id]="1"></student-detail>`
- `<student-detail [id]="999"></student-detail>` (cas “introuvable”)

😎 Critères de réussite (et 💡 indices...)

- Lecture correcte de l’id (appel du signal : `this.id()`).
- Gestion explicite du retour `Student | undefined` (TS strict respecté).
- Aucun accès direct à l’état interne du service (uniquement via `findById`)
- Encapsulation : `asReadOnly()` exposé, mutations via méthodes.
- Contrôle de flux `@if` pour le cas “introuvable”.
- `ChangeDetectionStrategy.OnPush` activé.

6. Exercice 14 : LoggingService + LogViewer

👉 Montrer l'utilisation d'un service basé sur une liste sans risque métier : un journal d'événements (logs) partagé par toute l'app.

1. Créez un type `LogEntry`

```
.ts
interface LogEntry {
  id: string      // identifiant unique (pour Angular trackBy) → crypto.randomUUID()
  ts: Date        // timestamp (date/heure du log) → new Date()
  message: string // contenu du log
  scope?: string  // (optionnel) origine du log (composant/service)
}
```

2. Créez un service `LoggingService` qui gère une liste de Log, à minima :

a. une propriété :

- `entries` qui expose la liste des logs en lecture seule

b. des méthodes

- `log(message: string, scope?: string)` : ajouter un log à la liste des logs
- `clear()` : effacer la liste des logs

3. Composant `LogViewerComponent` pour afficher les logs si on en a besoin :

- consomme `entries()` pour obtenir la liste des logs et l'afficher si il y en a au moins un, sinon n'affiche rien
- dispose d'un bouton "Reset" qui permet de supprimer l'ensemble des logs

4. ...

4. Démo d'usage → depuis n'importe quel composant, injecter le service et appeler :

- `logging.log('Student added', 'StudentList')`
- `logging.log('Promote clicked', 'StudentList')`
- Afficher `<log-viewer></log-viewer>` dans `AppComponent` (ou une sidebar).

😎 Critères de réussite

- Liste réactive de logs (`signal + asReadonly()`).
- Ajout et reset fonctionnels.
- Affichage performant (`@for + track`).
- Service injecté et utilisé depuis plusieurs composants (état partagé).

Bonus (facultatifs)💨

- Filtrage simple dans `LogViewer` par scope (input texte).
- `withComputed` dans un petit store de logs (compter par scope).
- Effet : persister les logs en `localStorage` (utiliser un `effect()`).

7. Récapitulatif et compétences acquises

Récapitulatif

- Un service Angular = classe TypeScript, injectée via DI.
- Les services centralisent la logique métier (composants = UI).
- `inject(Service)` = on récupère une instance (singleton).
- On peut simuler un backend via un service in-memory avant de passer à `HttpClient`.

Compétences acquises

- *Expliquer* l'intérêt des services.
- *Créer* et injecter un service Angular.
- *Déplacer* la logique métier hors des composants.
- *Utiliser* un service in-memory comme backend simulé.

8. Projet fil rouge : Festival

👉 On introduit les services dans le projet :

- Créer un `FestivalService` (en mémoire) pour stocker la liste des festivals.
- `AppComponent` injecte ce service et affiche la liste des festivals.
- Ajouter un bouton “+ Ajouter un festival” qui appelle `FestivalService.addFestival()`.

Plus tard, quand on verra `HttpClient`, on remplacera ce service in-memory par de vrais appels REST.



VII. Introduction à NgRx Signal Store

- ➔ *Expliquer* l'intérêt d'un store basé sur les signals pour gérer un état global ou partagé.
- ➔ *Créer et structurer* un `signalStore` avec `withState`, `withComputed` et `withMethods`.
- ➔ *Partager* un état entre plusieurs composants via l'injection
- ➔ *Mettre* en œuvre un `effect()` pour persister ou synchroniser l'état.

1. Pourquoi un Store ?

Les `signal()` suffisent pour un composant ou un parent/enfant.

Mais si plusieurs composants veulent partager le même état, il y a risque de duplication, et même de complexité.

Solution : un store centralisé (pattern inspiré de Redux, mais ici basé sur Signals).

Pour cela on utilisera la librairie *NgRx Signal Store*

```
# Installer NgRx Signal Store  
npm install @ngrx/signals
```

Quand mettre (ou pas) en Store ?

- Mettre en Store (global/scopé) : état transverse (ex: festival courant, utilisateur), état partagé entre vues.
- Ne pas mettre : grosses listes serveur sans stratégie d'invalidation (risque stale/memory) ou sinon à charger via service + cache TTL.

2. NgRx Signal Store : le principe

NgRx a introduit une nouvelle API : `signalStore` qui permet de créer un store basé sur Signals, simple et léger.

Il contient :

- un state défini par `withState`
- des méthodes, définies par `withMethods`, pour manipuler ce state.
- des computed, définis par `withComputed`,

👍 **Avantages :**

C'est comme un service Angular (injectable) et donc utilisable partout mais avec la réactivité automatique des `signal()`s.

3. Créer un Store Simple

On définit l'état de base :

```
.ts
export interface StudentListState {
  students: Student[]
  isLoading: boolean
  filter: { query: string; order: 'asc' | 'desc' }
}
```

On définit un état initial

```
.ts
const initialState: StudentListState = {
  students: [],
  isLoading: false,
  filter: { query: '', order: 'asc' },
}
```

On crée le store

```
.ts
export const useStudentListStore = signalStore(
  { providedIn: 'root' },
  withState<StudentState>(initialState),
  withComputed(({ students }) => ({
    studentsCount: computed(() => students().length),
  })),
  withMethods((store) => ({ // retourne un objet contenant les méthodes
    // construit ainsi l'API publique du store
    addStudent(s: Student) {
      patchState(store, patchAddStudent(s)) },
    removeStudent(id: number) {
      patchState(store, patchRemoveStudent({ id } as Student))
    },
    setFilter(partial: Partial<StudentListState['filter']>) {
      patchState(store, patchSetFilter(partial)) },
    setLoading(isLoading: boolean) {
      patchState(store, { isLoading }) },
    deleteAll() { patchState(store, { students: [] }) }
  })))
)
```

avec les méthodes de modifications :

```
.ts
function patchAddStudent(student: Student):
  PartialStateUpdater<{ students: Student[] }> {
  return (state) => ({ students: [...state.students, student] })
}
function patchRemoveStudent(student: Student):
  PartialStateUpdater<{ students: Student[] }> {
  return (state) => ({ students: state.students.filter(
    item => item.id !== student.id )
  })
}
function patchSetFilter(
  partial: Partial<StudentListState['filter']> ):
  PartialStateUpdater<{ filter: StudentListState['filter'] }> {
  return (state) => ({
    filter: { ...state.filter, ...partial } })
}
```

Explications :

- `useSignalStore(...)` → crée le store.
- `withState({ students: [] })` → initialise l'état (vide).
- `withMethods(...)` → ajoute des méthodes pour modifier le state.
 - `addStudent` : méthode pour ajouter un `student` à la liste.
 - `removeStudent` : méthode pour supprimer un `student` par son `id`.
- `PartialStateUpdater<{ students: Student[] }>` : type de fonction de modification de l'état du store pour simplifier l'écriture de la fonction de création du store



`students` est un bien signal, donc toute modification de `students` déclenche la réactivité.

Résultat

On a obtenu un service injectable (`useStudentStore`).

Ce service expose automatiquement et permet d'accéder à :

- `students` (un signal).
- `addStudent(...)` (méthode).
- `removeStudent(...)` (méthode).

👉 On peut utiliser `students` dans n'importe quel composant sans avoir besoin de réécrire ton état.



Un Signal Store, c'est comme un service Angular, sauf qu'il est déjà pensé pour stocker un état global avec des signals, et donc réactif par défaut.

4. Utiliser le Store dans un composant

👉 remplacer notre signal `students` déclaré localement par le `students` de notre store afin d'éviter à avoir le passer en `Input()` par exemple.

```
.ts
import { inject } from '@angular/core'
import { useStudentListStore } from '../store/student.store'

@Component({...})
export class StudentListComponent {
  store = inject(useStudentListStore)

  students = this.store.students // signal auto-exposé
}
```

```
.html
@for (f of students(); track s.id) { // Un signal se lit en l'appelant
  <student-card [name]="s.name" [promo]="s.promo"></student-card>
}
```

5. Syntaxe complète de `withMethods`

Dans l'exemple précédent, nous avons utilisé un `return` implicite.

La syntaxe complète permet plus de souplesse (initialisation, logique supplémentaire).

```
.ts
withMethods((store) => {
  // Zone d'initialisation (exécutée 1 seule fois)
  const initDate = new Date()
  // API publique du store
  return {
    addStudent(s: Student) { patchState(store, patchAddStudent(s)) },
    removeStudent(id: number) {
      patchState(store, patchRemoveStudent({ id } as Student))
    },
    deleteAll() { patchState(store, { students: [] }) },
    getInitDate() { return initDate }
  }
})
```



à retenir :

- `return` implicite = plus concis, souvent suffisant.
- `return` explicite = nécessaire si vous avez besoin d'initialisation ou de variables privées internes.

6. `effect()` dans un Store

Un `effect()` peut être déclaré dans la partie initialisation de `withMethods`.

Son rôle : déclencher une action dès qu'un signal du store change.

Exemple : persistance locale

```
withMethods((store) => {  
  // effet déclenché à chaque changement de representativeStudentId  
  effect(() => {  
    const id = store.representativeStudentId()  
    id ? localStorage.setItem('representativeStudentId', id)  
      : localStorage.removeItem('representativeStudentId')  
  })  
  return {  
    setRepresentative(id: string | null) {  
      store.representativeStudentId.set(id)  
    }  
  }  
})
```

🔑 Points clés :

- L'effet se relance automatiquement quand un signal utilisé dans son callback change.
- Très utile pour persister un état ou déclencher des actions côté client (log, synchro...).

7. Exercice 15 : Créer un StudentStore



👉 **Objectif** : réutiliser la base `StudentListService` pour centraliser l'état via un Signal Store.

1. Créez un fichier `student.store.ts`.
2. Définissez l'interface `StudentState` :

```
.ts
export interface StudentState {
  students: Student[]
}
```

3. Créez le store avec :
 - a. `useState<StudentState>({ students: [] })`
 - b. `withMethods` pour ajouter et supprimer un étudiant.
4. Injectez le store dans `StudentListComponent`.
5. Remplacez le signal local `students` par le store.
6. Vérifiez que l'ajout/suppression est bien réactif.

8. Service ou Store ?

Service + `signal<Student[]>`

Avantages👍

- Ultra simple à écrire et comprendre
- Pas besoin d'apprendre d'API supplémentaire
- Idéal pour des cas très basiques

Limites🤔

- Pas de structure standardisée → chaque service va gérer son state à sa façon.
- Si plusieurs signaux (students, isLoading, error, filter...), il faut tout gérer
- Pas de helpers intégrés pour les patterns courants (patch partiel, computed dérivés, entités CRUD).

Avantages 👍

- Standardisation : tous les stores ont la même structure (`withState`, `withMethods`, `withComputed`).
- Helpers intégrés :
 - `patchState` pour mettre à jour partiellement ton state
 - `withComputed` pour dériver des valeurs
 - `withEntities` pour CRUD d'entités (add/update/remove by id)
- Composable : il est possible de factoriser des morceaux et les réutiliser.

Inconvénients 🤔

- Légèrement plus verbeux qu'un service simple.
- Nécessite d'apprendre l'API `signalStore`, `patchState`, `withState`...
- Sur un petit projet ou un état ultra simple → "overkill".

👉 Une bonne règle d'utilisation

Service + signal si c'est juste du state local simple (ex. logs, un toggle, un compteur, ...)

Signal Store si c'est du state métier / feature (ex. gestion du user connecté, panier e-commerce, ...)

9. withEntities

`withEntities` est un outil de Signal Store qui :

- structure son state autour d'entités indexées par id
- expose des signaux tout prêts : `ids()`, `entities()`, `entityMap()`, `count()`, etc.
- fournit des helpers de mise à jour (updaters) : `addEntity`, `setEntity`, `removeEntity`, `updateEntity`, etc.

Exemple : .ts

```
export const StudentListStore = signalStore(  
  { providedIn: 'root' },  
  
  withEntities<Student>(), // Ajoute un state { ids, entities }  
  withMethods((store) => ({  
    // helper généré  
    addStudent(student: Student) { patchState(store, addEntity(student)) },  
    // remplace ou fusionne  
    updateStudent(student: Student) { patchState(store, updateEntity(student)) }  
    removeStudent(id: number) { patchState(store, removeEntity(id)) }  
  }))  
)
```


Le State créé automatiquement et géré par withEntities :

```
.ts
{
  ids: number[];
  entities: Record<number, Student>;
}
```

Et on obtient automatiquement :

- `store.entities()` → objet { 1: {...}, 2: {...} }
- `store.ids()` → [1, 2, 3]
- `store.count()` → nombre d'entités
- `store.entityMap()` → Map de tes étudiants

et des methodes déjà prêtes :

- `store.addEntity(student)`
- `store.setEntity(student)`
- `store.removeEntity(id)`
- `store.updateEntity(partial | projector)`

10. Récapitulatif

👉 Signal Store = service réactif structuré

- Centralise un état partagé (ex. festival courant, utilisateur).
- Injecté comme un service : `store = inject(useXStore)`.

Structure

- `withState` → définit l'état en `signal()`.
- `withComputed` → dérive des infos (ex. `isLoggedIn`, `userName`).
- `withMethods` → encapsule les mutations (ex. `setStudent`, `setUser`, ...).

Avantages

- Évite la duplication de l'état dans chaque composant.
- Permet à plusieurs composants indépendants d'utiliser la même source de vérité.
- Facilite les tests et la maintenabilité.
- Portée globale (App) mais peut être scopée par Route (cf un prochain cours)

Ex. fournir SubscriptionStore sur la route /promo/:id pour recréer/détruire l'état automatiquement à chaque accès à la promo.

Compétences acquises

- *Expliquer* pourquoi un store centralisé est utile.
- *Créer* un signalStore avec état + méthodes.
- *Consommer* le store dans un composant via `inject()`.
- *Partager* un état entre plusieurs composants grâce à NgRx Signal Store.

12. Projet Fil Rouge : Signal Store



👉 Mettre en place un Signal Store global pour gérer l'état partagé de l'application Festival :

- le festival courant sur lequel on travaille ;
- l'utilisateur connecté.

Cet évolution du projet permet de montrer comment plusieurs composants, sans lien direct, peuvent accéder au même état de manière réactive et synchronisée.

Cette évolution va se faire en plusieurs étapes...

Étape 0 : Interface User

Créez une interface `UserDto` pour un type `User` (il se semble pas nécessaire pour l'instant de disposer d'une class `User`).

Étape 1 : Créer un AppStateStore

1. État initial :

- `currentFestivalId`: `number` | `null`
- `currentUser`: `UserDto` | `null`

2. Ajoutez des `computed()` :

- `isLoggedIn` : vrai si un utilisateur est connecté.
- `userName` : nom de l'utilisateur ou "Invité".

3. Ajoutez des méthodes (`withMethods`) :

- `setFestival(id: string | null)`
- `setUser(user: User | null)`

Bonus 🌬️ : persistez `currentFestivalId` dans `localStorage` avec un `effect()`.

Étape 2 : Créez un composant Header

1. Le composant doit afficher :
 - L'utilisateur courant (nom)
 - Le festival courant (nom)
2. Ajoutez un bouton "next ➡" qui change le festival courant en prenant le prochain dans la liste des festivals (propriété locale)
3. Utilisez `setFestival()` du store pour changer l'Id

Étapes 3 : Créez un composant Sidebar

Le composant doit afficher dans un bloc qui a vocation à être latéral : "Festival courant : ..." en s'appuyant sur une liste locale pour retrouver le nom.

Ce composant ne doit pas connaître le composant Header

💡 Astuce : passer par le Store pour récupérer l'information

Étape 4 : Testez le partage d'état

1. Placez `Header` et `Sidebar` dans l'`AppComponent`
2. Vérifiez que :
 - quand on change de festival avec le bouton dans le Header, le Sidebar affiche immédiatement la même info
 - L'utilisateur courant est affiché dans les deux composants
3. Forcer un état vide (bouton Aucun → `setFestival(null)`) pour vérifier la robustesse

Résultat attendu

- Un seul store global gère `currentFestivalId` et `currentUser`.
- Plusieurs composants indépendants (`Header`, `Sidebar`) affichent les mêmes données partagées.
- Quand l'état change dans un composant, tous les autres sont mis à jour automatiquement.

Conclusion :



Le Signal Store permet de gérer un état global réactif et partagé, sans avoir besoin de passer les données en cascade par `input()` / `output()`.



VIII. Reactive Forms⁽¹⁾

- ➔ Understand the difference between template-driven forms and reactive forms (without detailing the former).
- ➔ Create a `FormGroup` and `FormControl` in a component.
- ➔ Bind a reactive form to the template with `[formGroup]` and `formControlName`.
- ➔ Handle value reading and submission.

1. Introduction to Angular forms

Angular offers 2 approaches:

- *Template-driven forms* (historical, weakly typed, not suitable for strict TS).
- *Reactive forms* (`FormGroup`, `FormControl`, more explicit, better for complex validations and reuse).

⇒ Reactive Forms should be preferred (official Angular recommendation).

2. Basics of Reactive Forms

FormControl

- Represents an individual field of the form (e.g. input, select).
- Contains the value of the field and its state (validity, touched, dirty, ...).

FormGroup

- Groups several `FormControl` (or other `FormControl`).
- Allows handling a complete form or a logical section as a single entity.

Simple Example simple : .ts

```
const name = new FormControl('')
const profile = new FormGroup({
  name: new FormControl(''),
  age: new FormControl(18),
});
```

Other notions (later):

FormArray

- Represents a dynamic list of FormControl or FormGroup.
- Useful for repetitive data (e.g. multiple addresses, multiple phones).

Validators

- Functions defining validation rules (e.g. required field, valid email).
- Can be synchronous or asynchronous.

3. Reading and handling values

Get a value (FormControl): `.value`

Example : `.ts`

```
const name = new FormControl('Alice') // typé string | null
console.log(name.value) // "Alice"
```

Get a value (FormGroup): `.get()` or `.value.name`

Legacy method

```
const profile = new FormGroup({
  name: new FormControl('Alice'),
  age: new FormControl(25),
})
console.log(profile.get(['name'])?!.value) // "Alice"
```

Modern method with typed form:

```
console.log(profile.value.name) // "Alice"
```



Prefer the modern method. Note that you can explicitly type:

```
new FormControl<string>('Alice') // string | null
```

Listening to value changes: `.valueChanges`

We're switching to asynchronous:

```
.ts
profile.valueChanges.subscribe(val => {
  console.log('Form updated :', val)
})
```

Handling errors manually: `.setErrors()`

`setError()` invalidates a form and associates an error object with the form's `.errors` property.

```
.ts
if (profile.controls.name.value === '') {
  profile.controls.name.setErrors({ required: true })
}
```

You can create your own error properties, e.g. `'incompatible'`. You can then query the error by:

```
.ts
form.errors?.['incompatible']
```

4. Example: simple form with FormGroup

app-student-form.ts

```
@Component({
  selector: 'app-student-form',
  templateUrl: './student-form.component.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
  imports: [ReactiveFormsModule],
})
export class StudentFormComponent {
  readonly form = new FormGroup({
    name: new FormControl('', { nonNullable: true }),
    year: new FormControl(2025, { nonNullable: true }),
    email: new FormControl('', { nonNullable: true }),
  });

  submit(): void {
    console.log(this.form.value);
  }
}
```

app-student-form.html

```
<form [formGroup]="form" (ngSubmit)="submit()">
  <input type="text" formControlName="name"
    placeholder="Nom" />
  <input type="number" formControlName="year"
    placeholder="Année" />
  <input type="email" formControlName="email"
    placeholder="Email" />
  <button type="submit">Enregistrer</button>
</form>
```


5. Exercise 16

👉 **Objective** : create a `SignupForm` component with:

- Three fields: username, password, email.
- A “Create account” button.

Instructions :

1. Create a `FormGroup` with 4 `FormControl` (`username`, `password`, `confirmPassword`, `email`).
2. In the template, bind fields with `[formGroup]` and `formControlName`.
3. On submission (`ngSubmit`), log `this.form.value` to the console.
4. Add a simple TypeScript rule: the form is valid only if `password === confirmPassword`.

💡 Tips :

1. `this.form.value` returns an object:

```
{ username: string, password: string, email: string }
```


Use `JSON.stringify()` for better console display.
2. A `FormGroup` exposes a `setErrors()` method. You can use it in a getter or a method (e.g. `submit()`) to invalidate the form if the two passwords differ.

6. FormBuilder: more concise syntax



FormBuilder est un service Angular qui simplifie la création de FormGroup, FormControl et FormArray. Il fournit des méthodes pratiques :

- `fb.group()`
- `fb.control()`
- `fb.array()`

sans FormBuilder

```
const profile = new FormGroup({  
  name: new FormControl(''),  
  age: new FormControl(18),  
  email: new FormControl('', [Validators.required, Validators.email]),  
})
```

avec FormBuilder

```
const fb = inject(FormBuilder)  
const profile = fb.group({  
  name: ['', { nullable: true }],  
  age: [18],  
  email: ['', { validators: [Validators.required, Validators.email] }],  
})
```

7. Mini-défi – Validation obligatoire

👉 Objectif : disposer d'un champ obligatoire

1. Refactorisez le formulaire `SignupForm` pour le créer avec un FormBuilder
2. Modifiez le formulaire `SignupForm` pour que :
 - Le champ `username` soit obligatoire.
 - Le champ email respecte le format email.
 - Le bouton “Créer compte” soit désactivé si le formulaire est invalide.

💡 Indices :

1. Un `FormControl` dispose d'une propriété `validators` qui contient un tableau de validateurs, vide par défaut. Lors de la création d'un `FormControl`, on peut donc lui fournir un tel tableau avec les validateurs que l'on souhaite avoir appliquer. Angular fournit par exemple `Validators.required` et `Validators.email`
2. Un `FormGroup` expose la propriété `invalid` que l'on peut utiliser pour savoir si le formulaire est conforme aux validateurs.

🖥️ Résultat attendu :

- Si le champ username est vide, le bouton de validation est désactivé.
- Si un nom est saisi, le bouton bouton est activé et si le form est validé, `form.value` est loggé.



Récapitulatif

- ➡ Un formulaire réactif Angular $\longleftrightarrow$ FormGroup + FormControl.
- Définition côté TS (`new FormGroup`).
- `FormControl` est un champ de formulaire (texte, email, nombre, etc.).
- `FormGroup` regroupe plusieurs contrôles $\rightarrow$ un formulaire complet.
- Lecture de l'état complet avec `this.form.value`.
- Liaison au template via `[formGroup]` et `formControlName`.
- Gestion d'un bouton de soumission avec (`ngSubmit`).
- Introduction des validations (`Validators.required`).

9. Compétences acquises

Compétences acquises :

- *Créer* un formulaire réactif `FormGroup` simple avec plusieurs `FormControl`
- *Associer* des champs du template (HTML) à un `FormGroup` via `formControlName`
- *Lire* et soumettre les valeurs.
- *Appliquer* des règles de base de validation (`Validators.required`, `Validators.email`).
- *Utiliser* la propriété `form.invalid` pour gérer l'activation/désactivation d'un bouton.
- *Comprendre* comment ajouter une logique simple de validation personnalisée (ex. comparaison de deux champs).



IX. Formulaires réactifs⁽²⁾

- ➔ Associer plusieurs validateurs à un champ.
- ➔ Afficher dynamiquement des messages d'erreurs dans le template.
- ➔ Créer des validateurs personnalisés simples (ex. comparaison).
- ➔ Ajouter dynamiquement des champs à un `FormGroup` ou `FormArray`.
- ➔ Intégrer des validations dans le projet fil rouge.

1. Valideurs intégrés et composition

Angular fournit de nombreux validators intégrés :

- `Validators.required`
- `Validators.email`
- `Validators.minLength(n)`
- `Validators.maxLength(n)`
- `Validators.pattern(regex)`

On peut combiner plusieurs validators dans un champ :

```
.ts
password: new FormControl('', {
  nullable: true,
  validators: [Validators.required, Validators.minLength(8)]
})
```

2. Afficher des messages d'erreurs

Chaque FormControl expose `.errors`, un objet clé-valeur.

Exemple :

```
.html
<input type="email" formControlName="email" />
@if (form.controls.email.errors?.['required']) {
  <small>Email obligatoire</small>
}
```

Chaque FormGroup expose `.invalid`, une propriété à true si un des validateurs échoue. Cela permet par exemple de "désactiver" le bouton de validation tant que le formulaire n'est pas complètement valide.

Bonne pratique :

Créer une fonction utilitaire

```
getErrorMessage(control: AbstractControl | null): string | null
```

pour centraliser la logique.

4. Exercice : Formulaire complet

👉 **Objectif** : améliorer `SignupForm` et/ou `StudentFormComponent` pour rajouter des vérifications, par exemple pour `SignupForm` :

- `username` : requis, min 3 caractères.
- `password` : requis, min 8 caractères.
- `confirmPassword` : doit être égal à `password`.
- `email` : requis, format email valide.

🖥️ Affichez les messages d'erreurs sous chaque champ si le champ est invalide et touché.

💡 **Indices** :

- `control.touched` permet de savoir si le champ a été modifié.
- avoir une méthode
`getErrorMessage(control: AbstractControl | null): string | null`
dans la classe peut aider

5. **FormArray** : gérer une liste dynamique de contrôles

Un **FormArray** est une collection de **FormControl** ou **FormGroup** :

- Utile pour les listes dynamiques (ex. ajouter plusieurs emails, plusieurs étudiants, plusieurs adresses...).
- Chaque élément du tableau est accessible par son index.

Exemple simple :

.ts : dans la classe du composant

```
readonly students = new FormArray<FormGroup>([]);

addStudent() {
  this.students.push(new FormGroup({
    name: new FormControl('', { validators: [Validators.required],
nonNullable: true }),
    email: new FormControl('', { validators: [Validators.email],
nonNullable: true }),
  }));
}
```

...

.html : dans le template du composant

```
<form [formGroup]="form">
  <div formArrayName="students">
    @for (student of students.controls; track $index) {
      <div [formGroup]="student">
        <input formControlName="name" placeholder="Nom" />
        <input formControlName="email" placeholder="Email" />
      </div>
    }
  </div>
  <button type="button" (click)="addStudent()">Ajouter étudiant</button>
</form>
```



À retenir :

- Quand utiliser **FormArray** : dès qu'on a une liste d'éléments homogènes.
- Différence avec **FormGroup** : **FormGroup** est une structure fixe, **FormArray** est une liste dynamique.
- Syntaxe spécifique dans le template : **formArrayName**, accès par index.
- Toujours typer les **FormArray** en **FormArray<FormGroup>** pour éviter les **any**.

6. Mini-défi : Formulaire dynamique avec **FormArray**



👉 **Objectif** : créer un composant **StudentGroupForm** qui permet d'ajouter plusieurs étudiants.

1. Définir dans le composant une propriété :
`readonly students = new FormArray<FormGroup>([ ])`
2. Créer une méthode `addStudent()` qui pousse un nouveau **FormGroup** avec deux champs :
 - name (requis),
 - email (email valide).
3. Dans le template, afficher tous les `students.control` (les 2 champs name et email)
4. Ajouter un bouton “Ajouter étudiant” qui appelle `addStudent()`.

🖥 **Résultat attendu** :

- L'utilisateur peut cliquer plusieurs fois sur “Ajouter étudiant”, à chaque fois de nouveaux champs apparaissent.
- Les règles de validation (required, email) s'appliquent à chaque étudiant ajouté.

💡 **Indice** : Utilisez `formArrayName` dans le template et `[formGroup]="student"` pour chaque élément.

7. Validateurs personnalisés

Un validateur est une fonction qui retourne :

- `null` si le contrôle est valide,
- un objet `{ errorName: true }` si invalide.

Angular fournit déjà des validateurs intégrés (required, email, minLength, etc.), mais on peut créer les nôtres pour des règles spécifiques.

Signature TypeScript officielle :

```
.ts  
(control: AbstractControl) => ValidationErrors | null
```

ou bien avec le type utilitaire Angular :

```
.ts  
export const myValidator: ValidatorFn = (control:  
AbstractControl) => { ... }
```

Exemple simple : comparer deux champs

Vérifier que password === confirmPassword

```
import { AbstractControl, ValidationErrors, ValidatorFn,
FormGroup } from '@angular/forms'

export const passwordMatchValidator: ValidatorFn = (control:
AbstractControl): ValidationErrors | null => {
  // le control est un FormGroup, on cast pour être sûr
  const group      = control as FormGroup
  const password   = group['password']?.value
  const confirm    = group['confirmPassword']?.value
  return password === confirm ? null : { mismatch: true }
}
```

8. Rendre un validateur personnalisé réutilisable

👍 **Bonne pratique** : placer vos validateurs dans un même répertoire, généralement `src/app/shared/validators/`.

🔍 **Exemple d'un validateur générique pour comparer deux champs :**

```
.ts
import { AbstractControl, ValidationErrors, ValidatorFn, FormGroup } from '@angular/
forms'

// comme on a besoin en paramètre des noms des champs,
// on définit une fonction qui retourne une fonction de type ValidatorFn
// construite à partir des noms de champs passés en paramètre
export function matchFieldsValidator( // on récupère les noms de champs concernés
  field: string,
  confirmField: string,
  errorKey = 'mismatch'
): ValidatorFn { // on retourne la fonction de type ValidatorFn
  return (control: AbstractControl): ValidationErrors | null => {
    const group = control as FormGroup
    const v1 = group[field]?.value
    const v2 = group[confirmField]?.value
    return v1 === v2 ? null : { [errorKey]: true }
  }
}
```

Dans le FormControl, il faudra penser à passer les paramètres.

9. Validateurs personnalisés : points clefs



À retenir

Un `ValidatorFn` reçoit toujours un `AbstractControl`.

En pratique, on pourrait avoir besoin de le caster :

- en `FormControl` si il est appliqué à un champ,
- en `FormGroup` si il est appliqué à un groupe

Le validateur doit être pur : ne pas modifier d'autres contrôles, juste retourner une erreur ou null.

Ne pas définir ses validateurs dans les composants, mais les ranger dans `shared/validators/`, voir exportez-les tous via un `index.ts`.

L'erreur est accessible dans le template via `.errors` (sur le `FormControl` ou le `FormGroup`).

10. Exercice 19 : Valideur personnalisé

👉 **Objectif** : Créer un valideur de champ qui impose que l'année soit ≥ 2000 , puis l'utiliser dans un formulaire réactif.

1. Écrire un valideur `yearMin2000Validator` (retourne `null` si valide, `{ yearTooSmall: true }` sinon).
2. L'appliquer à un `FormControl` `year` dans un `FormGroup`.
3. Dans le template, afficher un petit message d'erreur si le champ est invalide et touché.

💡 Indices

- Un valideur a la signature `(control: AbstractControl) => ValidationErrors | null`.
- On peut récupérer la valeur via `const value = control.value as number`.
- Le test doit être de la forme : `value >= 2000 ? null : { yearTooSmall: true }`.
- Pour le message d'erreur, tester : `form.controls.year.errors?.['yearTooSmall']` et `form.controls.year.touched`.

Bonus 🧠 :

Faites une version paramétrable : `minYearValidator(min: number)`



Récapitulatif

Un formulaire réactif peut intégrer des validations riches et dynamiques :

- Associer plusieurs validateurs intégrés (`required`, `minLength`, `email`, etc.).
- Exploiter `.errors`, `touched` et `dirty` pour afficher des messages pertinents.
- Gérer des listes dynamiques de champs avec `FormArray`.
- Créer et réutiliser des validateurs personnalisés (de champ ou de groupe).

Les validations sont déclaratives, centralisées dans le `FormGroup` pour un code clair et plus facilement maintenable.

12. Compétences acquises

Compétences acquises :

Appliquer plusieurs validateurs intégrés sur un champ.

Afficher des messages d'erreurs dynamiques en fonction de l'état du champ.

Construire un formulaire réactif dynamique avec `FormArray`.

Élaborer un validateur personnalisé de champ ou de groupe.

Organiser vos validateurs dans un répertoire partagé (shared/validators/).

Empêcher la soumission d'un formulaire invalide.

13. Projet fil rouge : Validation réactive



👉 **Objectif** : Étendre le formulaire `ExposantForm` (Festival) avec les contraintes suivantes :

- name : requis, min 3 caractères.
- type : requis (sélecteur).
- email : requis, format valide.

🖥️ **Résultat attendu**

- la soumission est empêchée tant que le formulaire est invalide
- les messages d'erreur dynamiques s'affichent

Bonus 🧠 :

Créez un validateur personnalisé `trimmedMinLength` qui a pour but d'éviter les noms constitués d'espaces et appliquer la longueur après trim (cas réel fréquent).

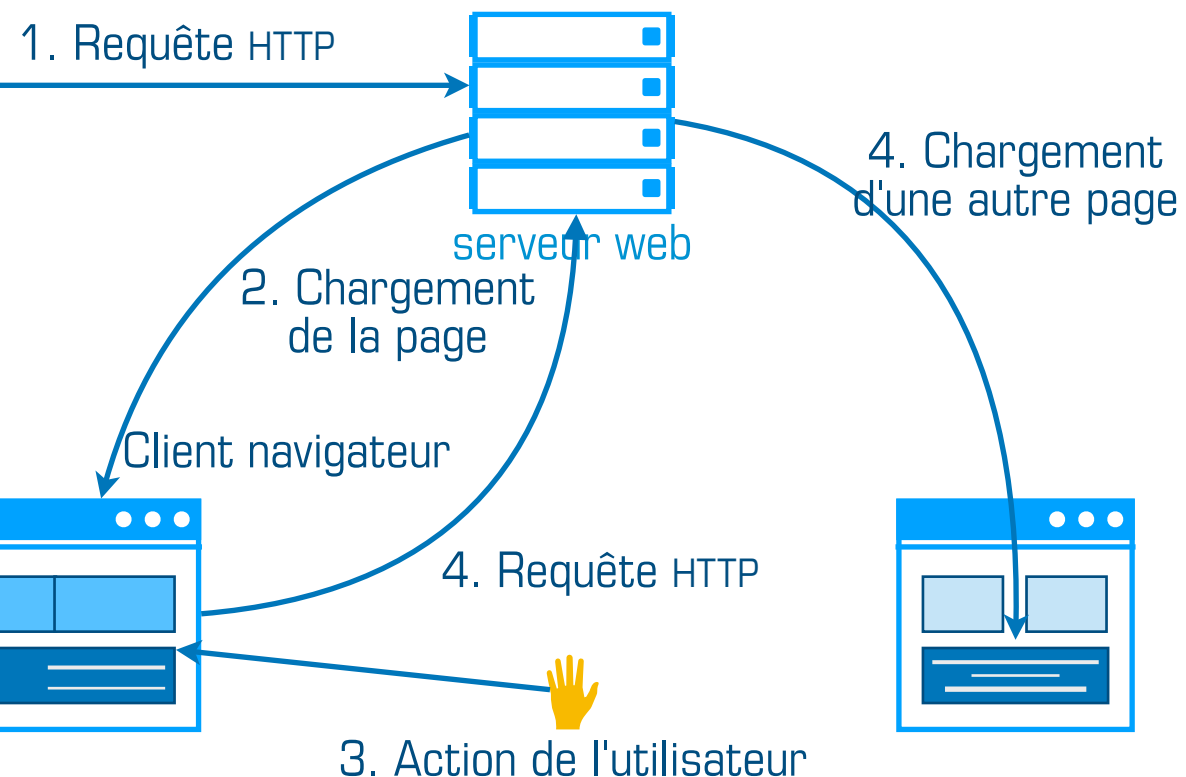


X. Routage Moderne⁽¹⁾

- ➔ Configurer des routes avec des composants autonomes.
- ➔ Utiliser `RouterOutlet` comme point d'ancrage.
- ➔ Naviguer avec `routerLink`.
- ➔ Organiser une application multi-vues (ex. Home / About).

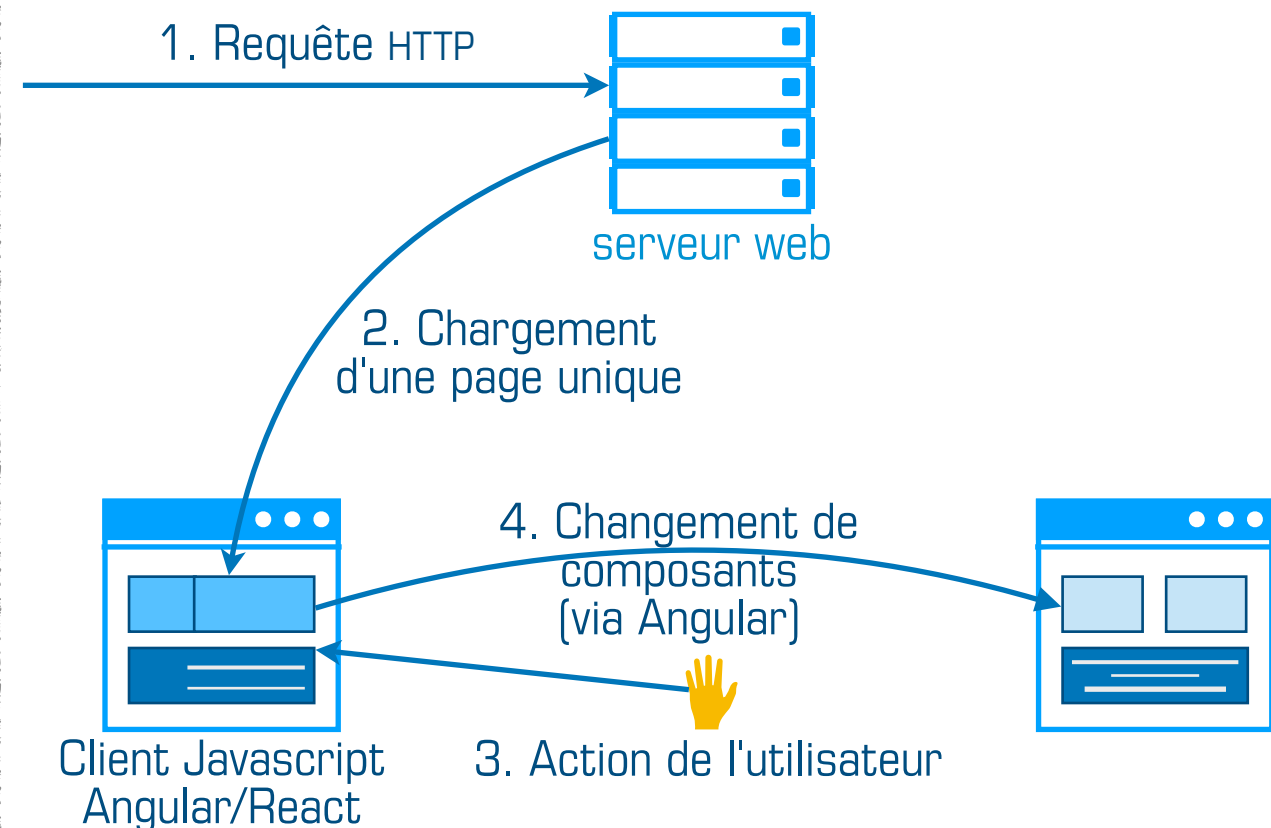
1. Navigation et Single Page Application

Angular fonctionne en Single Page Application.



Site classique (multi-page)

Multi-page → + SEO / – fluidité



Site moderne (Single Page Application)

SPA → + fluidité / – SEO initial

2. De la SPA au routage Angular

Angular fonctionne en SPA : une seule page HTML.

→ la navigation consiste donc à changer les composant dans la page pour donner l'impression de changer de page.

🤔 **Problème** : comment dire à Angular quel composant afficher quand l'URL change ?

👉 **Solution** : associer une URL à un composant

C'est ce que fait le Router Angular

3. Le principe du routage Angular

- RouterOutlet : zone dans laquelle Angular injecte le composant actif.
- Configuration : tableau de routes { path, component }.
- Navigation : changer de composant = changer de route (et donc d'URL).

Exemple :

```
/      → HomeComponent  
/about → AboutComponent
```

En SPA, changer de page revient en réalité à changer de composant actif.

4. Configuration de base

Trois fichiers travaillent ensemble :

1. `app.routes.ts` qui contient la liste des routes : chaque route associe un path (URL) à un composant.

 Exemple : `app.routes.ts`

```
import { Routes } from '@angular/router'
import { HomeComponent } from '../home/home.component'
import { AboutComponent } from '../about/about.component'

export const routes: Routes = [
  { path: '', component: HomeComponent }, // route par défaut
  { path: 'about', component: AboutComponent }
]
```

2. `app.config.ts` active le routeur avec `provideRouter(routes)`. C'est ici qu'on branche le tableau de routes à Angular.

 Exemple : `app.config.ts`

```
export const appConfig: ApplicationConfig = {
  providers: [ provideRouter(routes) ]
}
```


3. `main.ts` lance l'application avec la configuration (`appConfig`).

On n'a rien de spécial à écrire :

```
main.ts  
bootstrapApplication(AppComponent, appConfig)
```



Récapitulatif

- `app.routes.ts` définit les routes.
- `app.config.ts` active le routeur.
- `main.ts` démarre l'application.

Une fois cela fait, `<router-outlet>` affichera le composant de la route active...

5. AppComponent et le RouterOutlet

RouterOutlet

Balise spéciale d'Angular où sera affiché le composant correspondant à la route active.

Par exemple, si l' URL est /about, Angular affiche automatiquement **AboutComponent** dans le `<router-outlet>`.

Navigation avec routerLink

Directive Angular qui permet de changer de route sans recharger la page. Elle s'utilise sur des balises `<a>` ou `<button>`.

Exemple simple .html

```
<nav>
  <a routerLink="">Accueil</a>
  <a routerLink="about">À propos</a>
</nav>
<router-outlet></router-outlet>
```

!! Important :

- le composant qui contient l'outlet doit importer **RouterOutlet**
- le composant qui contient la nav doit importer **RouterLink**

6. Exercice 20: Home + About

1. Créer deux composants :
 - HomeComponent : titre + message de bienvenue.
 - AboutComponent : texte descriptif
2. Définir les routes '' et 'about'.
3. Placer un `<router-outlet>` dans AppComponent.
4. Ajouter un menu avec `routerLink` vers Home et About.

Résultat attendu

un clic sur un lien change la page sans rechargement complet.

7. Navigation avancée : lien actif & 404

routerLinkActive

Si cette directive est utilisée, lorsque le lien sera actif, une classe CSS spéciale sera appliquée à ce lien.

 Exemple .html

```
<a routerLink="about" routerLinkActive="active">À propos</a>
```

⇒ si l'URL courante est /about, Angular ajoute la classe active à ce lien (lien surligné).

 Une option utile : `[routerLinkActiveOptions]="{ exact: true }"`

Route par défaut (404)

Angular peut rediriger vers une page “NotFound” si l’URL ne correspond à aucune route.

 Déclaration .ts

```
{ path: '**', component: NotFoundComponent }
```

À toujours mettre en dernier dans `app.routes.ts`.

8. Exercice 21 : Navigation avancée

👉 Étendez votre application de l'exercice précédent

1. Lien actif : assurez-vous que le menu indique la page courante.
2. Ajoutez une Page 404 personnalisée

Bonus 🌩️ :

Stylisez la classe active pour mettre en évidence le lien.

9. Récapitulatif et Compétences acquises



Récapitulatif

- Le routage Angular relie une URL avec un composant via `RouterOutlet`.
- `provideRouter(routes)` pour configurer les routes.
- `routerLink` pour naviguer sans rechargement.
- une SPA reste une seule page HTML, Angular se charge d'afficher le bon composant.



Compétences acquises

Expliquer le rôle du routage dans une SPA.

Configurer des routes simples avec `provideRouter`.

Utiliser `RouterOutlet` pour afficher la vue courante.

Mettre en œuvre un menu de navigation avec `routerLink`.

Structurer une application Angular en plusieurs pages autonomes.

10. Projet fil rouge : page d'accueil + liste

Dans l'app Festival

1. Créez une page d'accueil (`FestivalHomeComponent`) affichant un message simple.
2. Créez une page `ExposantListComponent` (sera enrichie plus tard avec service + HTTP).
3. Configurez deux routes :
 - a. `/` → `FestivalHomeComponent`.
 - b. `/exposants` → `ExposantListComponent`.
4. Ajoutez un menu global avec Accueil et Exposants.

 **Résultat attendu :**

on pose les bases d'une app multi-pages.

11. Aller plus loin avec RouterOutlet

Routes imbriquées (child routes)

Un composant affiché dans le `<router-outlet>` principal peut lui-même contenir un autre `<router-outlet>`.

Cela permet d'avoir une navigation à plusieurs niveaux.

Exemple :

`/dashboard` → `DashboardComponent` (avec menu interne).

`/dashboard/profile` → `ProfileComponent` affiché dans le `<router-outlet>` du Dashboard.

Outlets nommés

Possibilité d'avoir plusieurs `<router-outlet>` avec des noms différents.

Sert à afficher plusieurs composants en parallèle.



Exemple :

.html

```
<router-outlet></router-outlet>                                <!-- principal -->
<router-outlet name="sidebar"></router-outlet><!-- secondaire -->
```

Route associée

.ts

```
{ path: 'chat', component: ChatComponent, outlet: 'sidebar' }
```



Résumé

- Cas simple : un seul outlet : suffisant dans 95 % des apps.
- Cas avancés :
 - Imbriqué : navigation hiérarchique.
 - Nommé : affichage parallèle (chat, sidebar, etc.).



XI. Routage moderne(2)

- Utiliser des routes paramétrées (`:id`).
- Récupérer les paramètres (`inject(ActivatedRoute)`)
- Mettre une navigation programmatique avec `router.navigate()`
- Implémenter un bouton retour.

1. Routes paramétrées : principe

Une route peut contenir une partie dynamique (un paramètre).

Le paramètre est noté `:nom` dans le chemin.

Exemple :

- `{path: 'students/:id', component: StudentDetailComponent }`
- URL : `/students/42` → Angular remplace `:id` par 42 et affiche `StudentDetailComponent`.

Intérêt :

On pourra récupérer ce paramètre dans le composant afin de d'afficher le détail d'un étudiant à partir par exemple d'un élément de liste (pattern liste/détail très courant).

Lier un paramètre avec routerLink

L'exemple précédent montre comment définir une route avec un paramètre. On a plus souvent besoin d'indiquer un paramètre dans un lien.

Dans le template, routerLink accepte un tableau de segments de route. Il reconstruit lui-même l'URL à partir de ces éléments.

On pourra ainsi lui indiquer des paramètres à insérer dans la route

Exemple dans StudentList :

```
.html
<a [routerLink]="['/students', student.id]">
  Voir détail
</a>
```

2. Récupérer un paramètre



Pour accéder à un paramètre comme `:id`, Angular fournit le service `ActivatedRoute`.

Ce service permet de récupérer toutes les informations à propos d'une route.

On peut l'injecter dans un composant et l'utiliser de deux façons pour récupérer les paramètres :

1. avec `snapshot` : simple et immédiat
2. avec `paramMap` qui est un observable

Avec snapshot

- Lecture immédiate du paramètre au moment où le composant est créé.
- Simple à utiliser.
- Ne réagit pas si l'URL change alors que le composant reste affiché.

Exemple :

```
.ts  
route = inject(ActivatedRoute)  
id = route.snapshot.paramMap.get('id')
```

Avec paramMap directement (observable)

- Lecture réactive : si l'URL change, la valeur est mise à jour sans recréer le composant.
- Plus flexible.

Exemple :

dans le composant :

```
.ts
route = inject(ActivatedRoute)
private id$ = this.route.paramMap.pipe(
  map(params => params.get('id')),
  filter((id): id is string => id !== null),
  map(id => Number(id))
)
public routeId = toSignal(this.id$, { initialValue: null })
```

dans le template :

```
.html
<p>ID étudiant : {{ id$ | async }}</p> // notez le $ et le async !
```



Résumé

- **Snapshot** → simple, adapté pour des cas basiques.
- **Observable** → réactif, recommandé quand la route peut changer sans recréer le composant.

3. Navigation programmatique



En plus des liens HTML (`routerLink`), Angular permet de déclencher une navigation directement en TypeScript.

Pourquoi faire ?

- Quand la navigation dépend d'une logique métier :
 - **Exemple** : après soumission d'un formulaire, rediriger vers une page de confirmation.
 - **Exemple** : si une condition n'est pas remplie, renvoyer vers la page d'accueil.
- Quand on veut offrir une navigation avec un bouton qui fait plus qu'un simple lien.

Comment ?

1. Injecter le service **Router**.
2. Appeler **router.navigate()** avec un tableau de segments d'URL.

.ts

```
import { Router } from '@angular/router';  
router = inject(Router);  
  
goToStudent(id: number) {  
  router.navigate(['/students', id])  
}
```



Résumé

- **routerLink** → navigation simple, déclarative dans le template.
- **router.navigate()** → navigation pilotée par la logique du composant.

4. Exercice 22: StudentDetail

👉 **Objectif** : mettre en place une route paramétrée et l'utiliser dans un composant de détail.

1. Ajoutez une route `{ path: 'students/:id', component: StudentDetailComponent }`.
2. Créez `StudentDetailComponent`.
3. Dans `StudentListComponent`, ajoutez un lien “Voir détail” sur chaque étudiant.
4. Dans `StudentDetailComponent`, récupérez l'ID avec `ActivatedRoute` et affichez-le.

😎 **Critères de réussite** :

- L'URL change en `/students/1`, `/students/2`, etc.
- `StudentDetailComponent` affiche bien l'ID récupéré.

5. Navigation retour avec Location

Angular expose le service Location (package @angular/common) qui permet de manipuler l'historique du navigateur. La méthode principale est `back()`

```
import { Location } from '@angular/common';
location = inject(Location);
goBack() {
  location.back()
}
```

!! `location.back()` ne fonctionnera que si l'utilisateur est arrivé sur cette page via une navigation précédente. On peut s'en prémunir :

```
goBack(): void {
  if (window.history.length > 1) {
    this.location.back()
  } else {
    this.router.navigate(['/default-route'])
  }
}
```

6. Exercice 23: bouton retour



👉 **Objectif** : ajouter un bouton “Retour” dans `StudentDetailComponent`.

1. Injectez le service `Location`.
2. Ajoutez un bouton `<button (click)="goBack()">Retour</button>`.
3. Implémentez `goBack()`

😎 **Critères de réussite** :

- Le bouton revient à la page précédente, quelle qu'elle soit.
- Si on a navigué depuis la liste → retour à la liste.
- Si on est arrivé directement en rentrant l'URL à la main → retour en arrière dans l'historique.

7. Récapitulatif & compétences acquises



Récapitulatif - Routage avancé :

- Définir des routes paramétrées (`:id`).
- Lire un paramètre avec `ActivatedRoute`.
- Naviguer en TypeScript avec `router.navigate()`.
- Implémenter un bouton retour.



Compétences acquises :

Créer une navigation liste $\longrightarrow$ détail.

Rendre les composants dépendants des paramètres de l'URL.

Mettre en place une navigation logique au-delà des liens statiques.

Justifier l'usage navigation programmatique dans un contexte métier

8. Projet Fil rouge

👉 **Objectif** : étendre l'application Festival avec un écran de détail.

1. Ajoutez une route
`{ path: 'exposants/:id', component: ExpositantDetailComponent }`.
2. Créez `ExpositantDetailComponent`.
3. Dans `ExpositantListComponent`, ajoutez un lien “Voir détail” sur chaque exposant.
4. Dans `ExpositantDetailComponent`, affichez le `nom`, le `type` ('éditeur', 'distributeur', 'partenaire', 'prestataire éditeur', 'boutique') et l'`email` de l'exposant.
5. Ajoutez un bouton “Retour à la liste” avec `router.navigate()`.

😎 **Critères de réussite** :

- L'URL `/exposants/...` affiche bien les détails.
- Le bouton “Retour” ramène à `/exposants`.



XII. Routage avancé

- Lazy loading avec `loadComponent`.
- Redirections (`redirectTo`).
- Gestion du titre des pages (`title`).
- Routes enfants (nested routes).
- Query parameters (`?key=value`).
- Guards (`CanActivate`).

1. Lazy loading avec loadComponent



Comme on est en SPA, lorsque les routes sont définies, généralement dans la page d'accueil, tous les composants sont chargés, ce qui peut alourdir le chargement initial de l'application

Le lazy loading permet de charger un composant seulement quand la route est appelée.

Exemple .ts

```
{  
  path: 'about', loadComponent: ()  
    => import('./about/about.component').then(m => m>AboutComponent)  
}
```

Doit-on privilégier le lazy loading ?

En général pour les premières pages ou les pages principales, on déconseille le lazy loading qu'on réservera à des pages plus accessoires

En effet, si le lazy loading améliore le chargement initial de l'application, il ajoute des requêtes qui peuvent alourdir les performances en particulier si on a des chargements imbriqués nombreux.

2. Redirection de routes

Une route peut rediriger automatiquement vers une autre.

Exemple classique : `/` $\longrightarrow$ `/home`.

```
.ts  
{ path: '', redirectTo: 'home', pathMatch: 'full' }
```



Note :

`pathMatch: 'full'` assure une correspondance exacte

C'est important car par défaut Angular traite les routes dans l'ordre et accepte les match partiel. Avec un chemin comme `' '`, il y a un risque de boucle.

3. Titre des pages

Angular permet à chaque route de définir un titre. Ce titre est par exemple utilisé dans les onglets des navigateurs.

```
.ts
{
  path: 'students',
  component: StudentListComponent,
  title: 'Liste des étudiants'
}
```

👍 **Bonne pratique :**

Toujours définir un titre dans les routes, ça permet d'aider l'utilisateur dans la navigation et ça améliore l'accessibilité.

4. Exercice 24

1. Mettre en place une redirection par défaut
 - a. Dans `app.routes.ts`, ajoutez une redirection
`/ → /students`.
 - b. Testez : accéder à `/` doit afficher la liste des étudiants.
2. améliorer l'UX en définissant des titres.
 - a. Ajoutez un titre pour vos routes principales (Home, Students, About).
 - b. Vérifiez dans le navigateur que l'onglet change selon la page.

5. Routes enfants

Les routes peuvent être hiérarchisées. Cela permet de gérer des sous-sections et d'afficher un composant enfant dans un composant parent en fonction de la route.

Le composant enfant s'affichera à la place de `<router-outlet>` dans le template du parent

Exemple : .ts

```
{
  path: 'admin',
  component: AdminComponent,
  children: [
    { path: 'users', component: UserListComponent },
    { path: 'settings', component: SettingsComponent }
  ]
}
```

Résultat attendu :

- `/admin/users` → Angular affiche `AdminComponent`, puis rend `UserListComponent` dans son `router-outlet`
- `/admin/settings` → idem avec `SettingsComponent`

6. Query parameters

Les query parameters sont des informations passées dans l'URL après ?.

Ils sont typiquement utilisés pour la pagination, filtres, tris, recherche.

Exemple : `/students?page=2&sort=asc`

Il y a deux manières de les lire :

1. Snapshot (lecture immédiate) : Simple, mais statique

Exemple : `.ts`

```
route = inject(ActivatedRoute)
page  = route.snapshot.queryParamMap.get('page')
```

2. Observable (lecture réactive) : réagit automatiquement si l'url change

Exemple : `.ts`

```
route = inject(ActivatedRoute)
page$ = route.queryParamMap.pipe(
  map(params => params.get('page'))
)
```

7. Guard de routes

Les Guard permettent de contrôler l'accès d'une route.

Un cas classique : empêcher l'accès à `/admin` si l'utilisateur n'est pas connecté et n'est pas un admin

Il faut définir une fonction de type `CanActivateFn` qui retourne un booléen et sera utilisée pour bloquer ou pas l'accès

```
.ts
import { CanActivateFn } from '@angular/router'
export const authGuard: CanActivateFn = () => {
  const router = inject(Router) // if not already injected
  const allowed = Math.random() > 0.5 // simulated true/false
  if (allowed) { return true }
  else { // indicate next navigation
    return router.createUrlTree(['/forbidden']) // no navigate inside navigation
  }
}
```

et l'utiliser avec `canActivate` pour protéger l'accès à la route :

```
.ts
{ path: 'about', component: AboutComponent, canActivate: [authGuard] }
```

 **Note** : il existe aussi `canActivateChild` pour protéger les routes enfants

8. Récapitulatif & compétences acquises



Récapitulatif : Routage avancé

- Charger un composant en lazy loading (`loadComponent`).
- Rediriger une route (`redirectTo`).
- Définir un titre de page (`title`).
- Organiser des routes hiérarchiques (`children`).
- Passer des infos via query params.
- Protéger une route avec un guard.



Compétences acquises :

Structurer et optimiser la navigation dans une SPA.

Améliorer l'expérience utilisateur avec titres et redirections.

Sécuriser et organiser l'application avec guards et routes enfants.

Mettre en œuvre le lazy loading pour optimiser le chargement initial.

Mettre en place une navigation imbriquée avec des routes enfants.

9. Projet fil rouge

👉 **Objectif** : enrichir l'application Festival avec des fonctionnalités avancées.

1. Ajoutez une redirection `/` $\longrightarrow$ `/exposants`.
2. Donnez un titre aux pages (Accueil, Exposants, Détail exposant).
3. Créez un composant `/admin` avec deux sous-routes (gestion exposants, gestion festivals).
4. Ajoutez un guard pour protéger `/admin`.



XIII. HTTP : Requêtes GET

- ➔ Découvrir le service `HttpClient`
- ➔ Comprendre comment effectuer une requête HTTP GET.
- ➔ Manipuler les données via :
 - ➔ Observables (`http.get<T>()`).
 - ➔ Signals (`httpResource`).
- ➔ Afficher les résultats dans un composant Angular.
- ➔ Appliquer le typage strict aux réponses.

1. Introduction : pourquoi HttpClient ?



Une application web fonctionne généralement avec un backend fournisseur des données métiers basé sur HTTP et une API REST.

En Angular, toutes les communications HTTP avec un backend passent par `HttpClient`.

C'est un service Angular prêt à l'emploi, basé sur des observables.

Ce service va permettre :

- une communication avec une API REST (JSON) ;
- de faire des requêtes GET, POST, PUT, DELETE ;
- de gérer headers, erreurs, et intercepteurs.

2. Préparer l'application

Avant d'utiliser HttpClient, il faut l'activer dans `app.config.ts` :

```
app.config.ts
import { provideHttpClient } from '@angular/common/http'

export const appConfig: ApplicationConfig = {
  providers: [provideHttpClient()]
}
```

Une fois le service activé, il peut être injecté partout.

 **Note :**

Pour le reste du cours, on utilisera pour simuler une API REST :

- `https://jsonplaceholder.typicode.com` (free fake API REST)
- `json-server` : un package node permettant de créer un serveur REST à partir d'un fichier json servant de base de données

3. Exemple de requête GET (Observable)

Consommation d'une API publique

Dans le composant

```
.ts
import { HttpClient } from '@angular/common/http';

const http = inject(HttpClient)
// méthode ancienne
post$ = http.get('https://jsonplaceholder.typicode.com/posts/1')
// en angular 20, on convertit en signal :
post = toSignal(
  this.http.get('https://jsonplaceholder.typicode.com/posts/1'),
  { initialValue: null }
)
```

Dans le template

```
.html
@if (post(); as p) {
  <pre>{{ p | json }}</pre>
} @else {
  <p>Chargement...</p>
}
```

4. Typage strict

On peut heureusement typer le retour de get, par exemple en any. Mais si on pourra accéder aux propriétés, on ne pourra pas contrôler le type et on risque d'avoir des bugs silencieux !

Le mieux est donc de typer les requêtes en définissant une interface :

```
.ts
interface Post {
  userId: number
  id: number
  title: string
  body: string
}

readonly data$ = this.http.get<Post>('https://
jsonplaceholder.typicode.com/posts/1')
```

+ Avantages :

- Vérification complète à la compilation
- Moins de bugs

5. Exercice 25

👉 **Objectif** : utiliser `HttpClient` pour récupérer et afficher des données.

1. Activez `HttpClient` via `provideHttpClient()`.
2. Créez un composant `PostListComponent`.
3. Dans ce composant, injectez `HttpClient` et récupérez les messages depuis : `https://jsonplaceholder.typicode.com/posts`.
4. Affichez la liste des posts dans le template avec `@for`.

🖥 **Résultat attendu :**

- Les titres des posts apparaissent correctement dans la page.

7. GET moderne (*Signal* avec `httpResource`)

`httpResource` a été stabilisé en Angular 20. Ce service permet de lier une requête HTTP à un `signal()`

Il remplace avec avantage l'approche observable dans les cas simples de CRUD :

- *Simplicité* : pas besoin de `| async` ni de RxJS
→ on lit la valeur directement avec `.value()`.
- *Réactivité native* : `httpResource` retourne un signal
→ se réévalue automatiquement.
- *Gestion intégrée des états* :
 - `isLoading()` : chargement en cours.
 - `error()` : erreur éventuelle.
 - `hasValue()` : y-a-t-il une valeurs ?
 - `value()` : données reçues.

Observable (<code>http.get</code>)	Signal (<code>httpRessource</code>)
Basé sur RxJS	Basé sur Signals
Retourne un <code>Observable<T></code>	Retourne un <code>Signal<T></code>
Nécessite <code>async</code> dans le template	Pas de <code>async</code> dans le template
Plus flexible (opérateurs)	Plus simple et intuitif



En résumé :

- `httpResource` est l'approche moderne à base de `signal()`.
- On continue d'utiliser `http.get()` + RxJS pour les flux complexes (ex. websockets, plusieurs sources de données) ou nécessitant l'utilisation d'opérateurs



Exemple : récupérer les posts (messages) avec httpResource

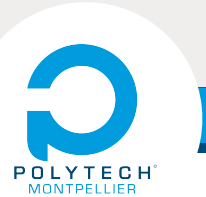
.ts

```
@Component({
  selector: 'post-list',
  standalone: true,
  templateUrl: './post-list.component.html',
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PostListComponent {
  readonly posts = httpResource<PostMessage[]>(
    { url: 'https://jsonplaceholder.typicode.com/posts' }
  )
}
```

.html

```
<h2>Posts</h2>
<ul>
  @for (p of posts.value(); track p.id) {
    <li>
      <strong>{{ p.title }}</strong><br>
      {{ p.body }}
    </li>
  }
</ul>
```

8. Gestion des états avec `httpResource`



Un `httpResource` ne donne pas seulement les données, il expose aussi son état :

- Chargement → `isLoading()`
- Succès → `value()`
- Erreur → `error()`

Cela permet d'éviter d'écrire un code compliqué avec RxJS pour gérer l'état d'une requête et de donner un feedback utilisateur immédiat :

- “Chargement en cours...” pendant la requête.
- Message d'erreur clair si l'API échoue.
- Affichage normal si tout va bien.

Comparaison avec `http.get` classique

Observable (<code>http.get</code>)	Signal (<code>httpResource</code>)
Nécessite un <code>Subject</code> ou <code>tap()</code> pour suivre l'état	<code>isLoading()</code> et <code>error()</code> intégrés
Gestion des erreurs manuelle (<code>catchError</code>)	Erreurs accessibles via <code>.error()</code>
Plus verbeux	Plus simple et lisible

Exemple: Gestion des états avec `httpResource`

```
.html
<h2>Posts</h2>

@if (posts.isLoading()) {
  <p>Loading...</p>
}

@if (posts.error()) {
  <p class="error">Error while loading posts</p>
}

@if (posts.hasValue()) {
  <ul>
    @for (p of posts.value(); track p.id) {
      <li>
        <strong>{{ p.title }}</strong><br>
        {{ p.body }}
      </li>
    }
  </ul>
}
```

9. Exercice 26

👉 **Objectif** : rendre l’affichage robuste.

Reprendre votre exercice précédent et

- Utilisez `httpResource` avec `PostMessage[]`.
- Ajoutez un message “Chargement...” quand la requête est en cours.
- Affichez title et body des posts si succès.
- Si erreur → affichez un message adapté.

😎 **Critères de réussite** :

- Code typé.
- Gestion de loading / success / error.

10. Récapitulatif & Compétences acquises

👉 Récapitulatif : Utiliser HttpClient :

- Importer via `provideHttpClient()`.
- Injection avec `inject(HttpClient)`.
- `http.get<T>()` retourne un `Observable` utilisable avec `| async` dans le template.
- `Signal(httpResource)` : moderne, simple, gère loading/error/success.
- Toujours typer la réponse avec une interface.

🔑 Compétences acquises

Configurer `HttpClient` dans une application Angular.

Effectuer une requête GET vers une API REST.

Afficher des données asynchrones avec `Observables`.

Utiliser des `signal()` pour récupérer les données avec `httpResource`

Garantir un code robuste grâce au typage strict.

11. Project fil rouge

👉 **Objectif:** connecter l'application Festival à un backend fictif.

1. Créez une base de donnée et une API REST avec `json-server` qui contienne une liste d'exposants
2. Activez `HttpClient` dans l'application Festival.
3. Créez un `ExposantService` qui appelle votre API REST (ex. `/api/exposants`).
4. Dans `ExposantListComponent`, consommez ce service pour charger la liste des exposants.
5. Affichez la liste des exposants avec `@for`.

🖥️ **Résultat attendu :**

La liste des exposants de la base s'affiche dans le composant list

Bonus 🌪️ :

Rajoutez un exposant dans la base grâce à un formulaire de saisie d'un nouvel exposant et vérifiez qu'il s'affiche dans la liste

12. Défi Fil Rouge



Chercher un jeu via api boardgamegeek et former le type pour le mettre dans un backend



XIV. HTTP – POST & intégration avec Signals

- ➔ Comprendre comment envoyer des données à une API avec `HttpClient`.
- ➔ Utiliser `http.post<T>()` avec typage strict.
- ➔ Combiner formulaires réactifs et requêtes POST.
- ➔ Intégrer le résultat dans un état réactif via `signal()`.
- ➔ Mettre à jour l'affichage après insertion de données.

1. Envoyer des données avec HttpClient (POST)



Une requête POST sert à envoyer des données vers un serveur (ex. formulaire, ajout d'un objet).

Le serveur peut :

- enregistrer les données (base de données),
- et renvoyer en réponse l'objet créé (souvent avec un id).

`HttpClient` fournit la méthode `http.post<T>(url, body)` où

- `url` = endpoint de l'API REST,
- `body` = données envoyées (souvent JSON),
- `T` = type attendu en retour.

Exemple

```
.ts
const http = inject(HttpClient);

addPost(post: Post) : Observable<Post> {
  return http.post<Post>(
    'https://jsonplaceholder.typicode.com/posts',
    post
  )
}
```



Résumé :

- `.post()` sert à envoyer des données via une requête POST
- Avec Angular, c'est typé : on sait exactement ce que le serveur doit renvoyer.
- La réponse peut être utilisée pour mettre à jour l'interface (liste, confirmation, etc.)

2. POST moderne avec Signals et httpResource



Avec `http.post()`, on reçoit un `Observable`. Il faut donc y souscrire (`subscribe`) pour utiliser la réponse.

Avec un `signal()`, on peut :

- conserver l'état local (liste, objet courant),
- mettre à jour automatiquement l'affichage après un POST réussi.

Avantages de cette approche

- *Réactivité native* : le composant est mis à jour dès qu'on modifie le signal.
- *Simplicité* : plus besoin de `| async` ni de gérer manuellement les Observables.
- *Cohérence* : s'intègre avec la logique déjà vue pour `httpResource` et `signal()`.

Exemple

```
.ts
posts = signal<Post[]>([]) // état local

function addPost(post: Post) {
  http.post<Post>('https://jsonplaceholder.typicode.com/posts',
post)
  .subscribe(newPost => {
    posts.update(arr => [...arr, newPost])
  })
}
```

Explications

- On envoie un post à l'API.
- Le serveur renvoie l'objet créé.
- À la réception (`.subscribe`), on met à jour `posts` via `.update()`.
- Le template réagit automatiquement.

Résumé :

- Avec `signal()`, on combine requête POST + mise à jour locale.
- Cela permet de voir immédiatement l'effet côté UI, sans recharger ni resynchroniser toute la liste.
- C'est la méthode recommandée pour des formulaires simples qui modifient une collection en mémoire.

3. Exercice 27

👉 **Objectif** : insérer un nouvel objet et l'afficher automatiquement.

1. Créez un composant `PostFormComponent`.
2. Ajoutez un `signal<Post[]>` pour stocker les posts.
3. Dans votre composant `PostListComponent`, rajoutez la possibilité d'ajouter un post
4. Lorsqu'un post est ajouté → mettez à jour le signal avec `.update()`.

😎 **Critères de réussite** :

- La liste s'agrandit sans rechargement.
- Les posts existants + le nouveau sont visibles.

4. Pourquoi encapsuler HTTP dans un service ?



Séparation des responsabilités :

- Le composant gère l'UI (affichage, interactions).
- Le service gère l'accès données/HTTP (URL, headers, erreurs).

Réutilisabilité & DRY : un même appel (ex. “créer un Post”) peut être utilisé par plusieurs composants.

Testabilité : on mock le service dans les tests de composants (pas besoin d'appeler l'API réelle).

Typage & contrat : le service expose une API claire et typée (ex. `create(dto: PostCreate): Observable<Post>`).

Évolutivité : centraliser

- la baseUrl,
- la gestion d'erreurs,
- la sérialisation/désérialisation (mapping),
- et plus tard les intercepteurs (auth, logs, retry).

5. Deux patterns de service possibles (Angular 20)



1. “*API pure*” (méthodes HTTP)

- Le service n’expose que des méthodes qui renvoient des *Observables*
- L’état (liste locale, loader) est géré dans le composant.

2. “*Service avec état* (Signals)”

- Le service gère l’état (liste, loading, error) avec signals ;
- Le composant consomme les signals en lecture seule (`asReadonly()`).

Lequel Utiliser ?

- *API pure* : démo, petites pages, logique état simple côté composant.
- *Service avec état* : Pages multi-composants, besoin de partager l’état, éviter la “plomberie” dans l’UI.

Exemple Service API Pure

post.service.ts

```
@Injectable({ providedIn: 'root' })
export class PostService {
  private readonly http = inject(HttpClient)
  private readonly base = 'https://jsonplaceholder.typicode.com/posts'

  list() : Observable<Post[]> {
    return this.http.get<Post[]>(this.base)
  }

  create(dto: PostCreate) {
    return this.http.post<Post>(this.base, dto)
  }
}
```

Consommation côté composant (extrait) :

- Liste : `posts$ = postService.list()` (Observable + | async)
- Création : `postService.create(form.value).subscribe(
/* mise à jour UI */)`

Exemple Service avec état (Signals)

```
post-store.service.ts
@Injectable({ providedIn: 'root' })
export class PostStore {
  private readonly http = inject(HttpClient)
  private readonly base = 'https://jsonplaceholder.typicode.com/posts'

  private readonly _posts = signal<Post[]>([])
  readonly posts = this._posts.asReadonly()

  loadAll() : void {
    this.http.get<Post[]>(this.base).subscribe(data => this._posts.set(data))
  }

  add(dto: PostCreate): Observable<Post> {
    this.http.post<Post>(this.base, dto).subscribe(newPost => {
      this._posts.update(list => [newPost, ...list])
    })
  }
}
```

Composant :

- `store.loadAll()` au init ;
- Template : `@for (p of store.posts(); track p.id) { ... } ;`
- Submit : `store.add(form.value).`

Exemple Service avec état (httpResources - version simple)

httpResource gère le fetch réactif (value, isLoading, error).

```
post-store.service.ts
@Injectable({ providedIn: 'root' })
export class PostService {
  private readonly postsResource = httpResource<Post[]>(() => '/api/posts')

  readonly posts : Signal<Post[]> = this.postsResource.value // signal
  readonly isLoading : Signal<boolean> =
    this.postsResource.isLoading // signal
  readonly error : Signal<unknown> = this.postsResource.error // signal

  load(): void { this.postsResource.get() }

  add(dto: PostCreate): Observable<Post> {
    return this.http.post<Post>('/api/posts', dto)
    // usage: service.create(dto).subscribe(() => service.load())
  }
}
```

À noter la nécessité pour le composant de recharger la liste après un add (cf commentaire dans le code)

Exemple Service avec état (httpResources + mise à jour)

httpResource gère le fetch réactif (value, isLoading, error).

Après une mutation (add, delete...), on relance un load() pour garantir la cohérence avec le backend.

```
post-store.service.ts
@Injectable({ providedIn: 'root' })
export class PostService {
  private readonly base = '/api/posts';
  private readonly http = inject(HttpClient)
  private readonly postsResource = httpResource<Post[]>(() => base)

  readonly posts : Signal<Post[]>      = this.postsResource.value      // signal
  readonly isLoading : Signal<boolean> = this.postsResource.isLoading // signal
  readonly error : Signal<unknown>     = this.postsResource.error      // signal

  load(): void { this.postsResource.get() }

  add(dto: PostCreate): Observable<Post> {
    return this.http.post<Post>(base, dto).pipe(
      tap(() => this.load()), // ne s'exécute que si le POST réussit
      // laisse la gestion d'erreur au composant appelant
      catchError((err: unknown) => throwError(() => err))
    )
  }
}
```

Bilan :

- Simplicité : un seul endroit pour toute la logique réseau.
- Clarté : composants = uniquement de la lecture des signals.
- Robustesse : après chaque mutation, rechargement depuis le backend $\implies$ pas de divergence, même en multi-utilisateurs.

6. PUT et DELETE

HttpClient fournit aussi d'autres méthodes pour mettre à jour ou supprimer des ressources.

```
.ts
// PUT → remplacer (mise à jour complète)
http.put<Post>(`/api/posts/${id}`, updatedPost)

// PATCH → mise à jour partielle (pas toujours supportée par le backend)
http.patch<Post>(`/api/posts/${id}`, { title: 'Nouveau titre' })

// DELETE → suppression d'une ressource
http.delete<void>(`/api/posts/${id}`)
```



Récapitulatif :

Même usage que POST : `<T>` où T est le type de retour attendu.

- PUT → remplace entièrement l'objet.
- PATCH → mise à jour partielle.
- DELETE → pas de body dans la requête, retourne généralement void.

6. Exercice 29 : créer et utiliser un service HTTP

👉 **Objectif** : Mettre en place un service qui gère une ressource **Post** (liste et création) puis l'utiliser dans deux composants **PostListComponent** pour afficher la liste, et **PostFormComponent** pour ajouter un nouvel élément.

1. Typage strict : définir **Post** et **PostCreate** (pas de **any**).
2. Créez un **PostService** qui
 - a. charge la liste des posts depuis **/api/posts** et expose :
 - la liste des posts,
 - l'état de chargement,
 - l'erreur éventuelle.
 - b. Le service doit permettre d'ajouter un nouveau post. Après un ajout réussi, la liste affichée doit être à jour (cohérente avec le backend).

3. Créez un composant PostFormComponent qui doit
 - a. afficher un formulaire réactif (au minimum : title, body) ;
 - b. empêcher les envois multiples pendant la soumission (désactivation du bouton ou équivalent) ;
 - c. afficher un message d'erreur lisible en cas d'échec serveur ;
 - d. En cas de succès, réinitialiser le formulaire.
4. Créez un PostListComponent qui doit
 - a. afficher la liste des posts fournie par le service ;
 - b. afficher un état "Chargement..." quand la récupération est en cours ;
 - c. Afficher un message d'erreur si le chargement échoue ;
 - d. Utiliser un track pertinent pour la liste.

5. La page principale (exigence UX) doit afficher le formulaire et la liste sur la même vue (même page) pour constater immédiatement la mise à jour après un ajout.

Contraintes générales

- Composants Angular modernes : standalone, OnPush, imports explicites.
- Pas de HttpClient dans les composants (uniquement dans le service).
- Pas de any ; types Post, PostCreate requis.
- Utiliser la nouvelle syntaxe de template (@if, @for) et le typage strict.



Critères de réussite

- Au premier affichage, la liste est chargée depuis l'API et visible.
- Lorsqu'on crée un post via le formulaire :
- le bouton d'envoi est désactivé pendant l'appel,
 - en cas de succès, la liste se met à jour sur la même page,
 - en cas d'échec, un message d'erreur est affiché clairement.
- Le code des composants ne contient aucun appel HTTP.

Bonus 🌩️ (facultatif) :

- Ajouter un compteur dynamique “N posts” au-dessus de la liste.
- Ajouter un bouton “Recharger” qui force le rechargement depuis l’API.
- Ajouter une suppression d’un post dans le service et dans la liste.

7. Récapitulatif



Récapitulatif HttpClient

- `HttpClient` = API bas niveau pour requêtes GET/POST/DELETE
- `httpResource()` = simplifie les requêtes GET réactives et expose des signals `value()`, `isLoading()`, `error()`.
- Pattern recommandé :
 - Encapsuler l'accès API dans un service injectable,
 - Exposer les signals (`posts`, `isLoading`, `error`) aux composants,
 - Refetch (`.get()`) après une mutation pour garder la liste cohérente avec le backend,
 - Aucune logique HTTP dans les composants (principe de séparation des responsabilités).
- Gestion UX :
 - Signal `submitting` pour désactiver le bouton,
 - `finalize` pour réinitialiser l'état après succès ou erreur,
 - Message d'erreur affiché dynamiquement.

8. Compétences acquises



Configurer et utiliser `HttpClient` pour communiquer avec une API REST.

Utiliser `httpResource()` pour des requêtes GET réactives basées sur les signals.

Créer un service Angular qui encapsule l'accès API et expose des signals (`posts`, `isLoading`, `error`).

Assurer la cohérence entre le frontend et le backend grâce au rechargement (`load()`) après mutation.

Construire des composants réactifs qui consomment un service

Respecter les bonnes pratiques Angular

9. Projet Fil Rouge

👉 **Objectif** : intégrer la persistance côté Festival.

1. Créer un service `ExposantService` qui encapsule la gestion des exposants :
 - `load()` pour récupérer la liste (`HttpRequest`),
 - `add()` pour envoyer un nouvel exposant (`HttpClient.post`) et recharger la liste.
2. Exposer : `exposants()`, `isLoading()`, `error()`.
 - Adapter `ExposantFormComponent` :
 - Formulaire réactif avec champs `name`, `type`, `email`.
 - À la soumission → appeler `ExposantService.add()`.
 - Afficher un état `submitting` et un message d'erreur en cas d'échec.
3. Adapter `ExposantListComponent` :
 - Afficher la liste réactive via `ExposantService.exposants()`.
 - Gérer l'état "Chargement..." et "Erreur".
 - Vérifier que la liste se met bien à jour après un ajout.
4. Disposition : afficher formulaire + liste sur la même vue pour observer la mise à jour immédiate.



XV. HTTP Intercepteurs & Sécurité

- ➔ Expliquer le rôle d'un intercepteur (`HttpInterceptor`) dans Angular.
- ➔ Mettre en œuvre un intercepteur pour ajouter un header (ex. JWT).
- ➔ Implémenter un intercepteur de log / gestion des erreurs.
- ➔ Intégrer un intercepteur d'authentification dans l'application Festival.

1. Qu'est-ce qu'un intercepteur HTTP ?



Un intercepteur est un maillon du pipeline `HttpClient` qui peut lire/modifier chaque requête sortante et réponse entrante avant qu'elles n'arrivent à ton code.

Il s'applique de façon transversale à toutes les requêtes : pas besoin de répéter la logique dans chaque service.

Cas d'usage :

- *Auth* : ajouter automatiquement un header `Authorization: Bearer <token>`.
- *Erreurs globales* : journaliser, afficher un toast, rediriger /login en 401.
- *Observabilité* : log/mesure de la durée des appels.
- *Entêtes communs* : Accept-Language, X-Feature-Flag, etc.

Principes forts à respecter :

- *Immutabilité* : on ne modifie pas la requête, on la clone.
- *Transparence* : on passe la main avec `next.handle(...)`.
- *Sobriété* : pas de logique métier lourde dans un intercepteur.

2. Pipeline & cycle de vie



Chaîne d'intercepteurs

- Aller (request) : appliqués dans l'ordre où tu les declares.
- Retour (response) : les réponses repassent en sens inverse (dernier déclaré → premier servi).
- Conséquence : l'ordre dans `withInterceptors([...])` est important.

Immutabilité & clonage

- `HttpRequest` est immutable → pour modifier URL/headers/body :
`req.clone({ setHeaders: {...} })`.

Ne pas bloquer la chaîne

Toujours retourner `next.handle(reqModifié)` (ou req si inchangé).

Pour observer la réponse → opérateurs RxJS (`tap`, `catchError`, `finalize`).

3. Intercepteurs de réponse

Un intercepteur peut agir uniquement sur la réponse :

- Transformation des données en sortie :
 - Ex. uniformiser les dates $\longrightarrow$ `string` $\longrightarrow$ `Date`.
 - Ex. extraire data d'un objet JSON `{ data: ... }`.
- Centraliser les post-traitements pour éviter la duplication.

Exemple :

```
export const DateInterceptor: HttpInterceptorFn = (req, next) =>
  next(req).pipe(
    map(event => {
      if (event instanceof HttpResponse) {
        event = event.clone({ body: normalizeDates(event.body) })
      }
      return event
    })
  )
```

Cela permet aux composants/services de travailler avec des données déjà propres.

Souvent combiné avec un intercepteur d'erreurs pour fiabiliser les réponses.

4. Enregistrement & portée

Où déclare-t-on les intercepteurs ?

- Dans la config de l'app (standalone) via `provideHttpClient(withInterceptors([ ... ]))`.
- Ils s'appliquent à ce scope d'injection (généralement l'app entière).

Deux styles d'intercepteurs

- Classe (historiquement) : implements `HttpInterceptor` $\rightarrow$ `intercept(req, next)`.
- Fonction (moderne) : `HttpInterceptorFn = (req, next) => ...` (souvent plus léger).

Bonnes pratiques

- Intercepteurs stateless (pas d'état persistant).
- Pas de subscribe dans l'intercepteur $\rightarrow$ on renvoie l'Observable.
- Éviter les effets de bord non idempotents.

Configuration

```
app.config.ts
providers: [ provideHttpClient(withInterceptors([authInterceptor, errorInterceptor]))
]
```

5. Erreurs & logs (principe)

But : Uniformiser le comportement en cas d'erreur : log, message utilisateur, redirection si besoin.

Stratégie typique

- Laisser passer la requête : `next.handle(req)`
- Ajouter en retour (`pipe`) :
 - `tap` pour loguer le succès (facultatif),
 - `catchError` pour enrichir/loguer/convertir l'erreur,
 - `finalize` pour mesurer la durée.

Décisions UX

- 401 : rediriger `/login` (si app authentifiée).
- `>= 500` : afficher un toast "Erreur serveur".
- Timeout : message spécifique.

Exemple 2 — Logs + durée + erreurs

```
.ts
export const loggingInterceptor: HttpInterceptorFn = (req, next)
=> {
  const t0 = Date.now()
  return next(req).pipe(
    tap(() => console.log('HTTP OK:', req.method, req.url)),
    catchError(err => {
      console.error('HTTP ERR:', req.method, req.url, err.status)
      return throwError(() => err)
    }),
    finalize(() => console.log('HTTP', req.method, req.url,
'durée=', Date.now() - t0, 'ms'))
  )
}
```

6. Exercice guidé 31: Intercepteur de log



👉 **Objectif** : Suivre toutes les requêtes HTTP pour mieux comprendre le pipeline.

1. Créer un intercepteur `LoggingInterceptor`.
2. Chaque requête doit afficher dans la console :
 - La méthode (GET/POST/DELETE...).
 - L'URL appelée.
 - Le statut final (succès ou erreur).
3. Mesurer et afficher le temps total d'exécution (en ms).

💡 **Indice** :

- Utiliser `Date.now()` avant et après.
- `finalize()` de RxJS permet d'exécuter du code une fois la requête terminée (succès ou échec).

7. Exercice 32 : Gestion globale des erreurs

👉 **Objectif** : Améliorer l'expérience utilisateur en centralisant la gestion des erreurs.

1. Créer un intercepteur `ErrorInterceptor`.
2. Ajouter un comportement selon le code HTTP:
 - 401 : afficher un message "Authentication requise" (+ bonus : rediriger vers /login)
 - 500 ou plus : afficher "Erreur serveur" (par un `alert()` ou `console.error()`)
 - Autres erreurs : logger dans la console pour le debug
3. Vérifier en testant sur une API volontairement incorrecte (`/api/bad-url`).

💡 **Indice** :

- Utiliser `catchError()` dans l'intercepteur pour capturer l'erreur.
- `err.status` contient le code HTTP.

8. AuthInterceptor : Ajouter un JWT



But : Joindre un token à toutes les requêtes sortantes vers l'API.

Points clés

- Le token vient d'une source sûre (p.ex. AuthService / localStorage).
- On exclut au besoin certaines URLs (ex. /assets, /auth/login).
- On clone proprement la requête : `setHeaders:`
`{ Authorization: 'Bearer ...' }.`

Erreurs fréquentes à éviter

- Oublier le `clone()` → ne marche pas.
- Surcharger des headers existants.
- Intercepter aussi les appels vers des domaines qui ne doivent pas être authentifiés.

Exemple : Auth

```
.ts
export const authInterceptor: HttpInterceptorFn = (req, next) =>
{
  const isApi = req.url.startsWith('/api/')
  const token = localStorage.getItem('token')
  if (!isApi || !token) return next(req)

  const authReq = req.clone({ setHeaders: { Authorization:
`Bearer ${token}` } })
  return next(authReq)
}
```

!! En pratique, stocker les tokens dans `localStorage` présente des risques de sécurité. On le fait ici pour simplifier.

9. Ordre des intercepteurs

Ordre d'exécution = ordre d'enregistrement

- Les intercepteurs sont appliqués dans la séquence déclarée dans `withInterceptors([ ... ])`.
- Conséquence : **AuthInterceptor** peut agir avant que **LoggingInterceptor** logge la requête → ou l'inverse selon la configuration.

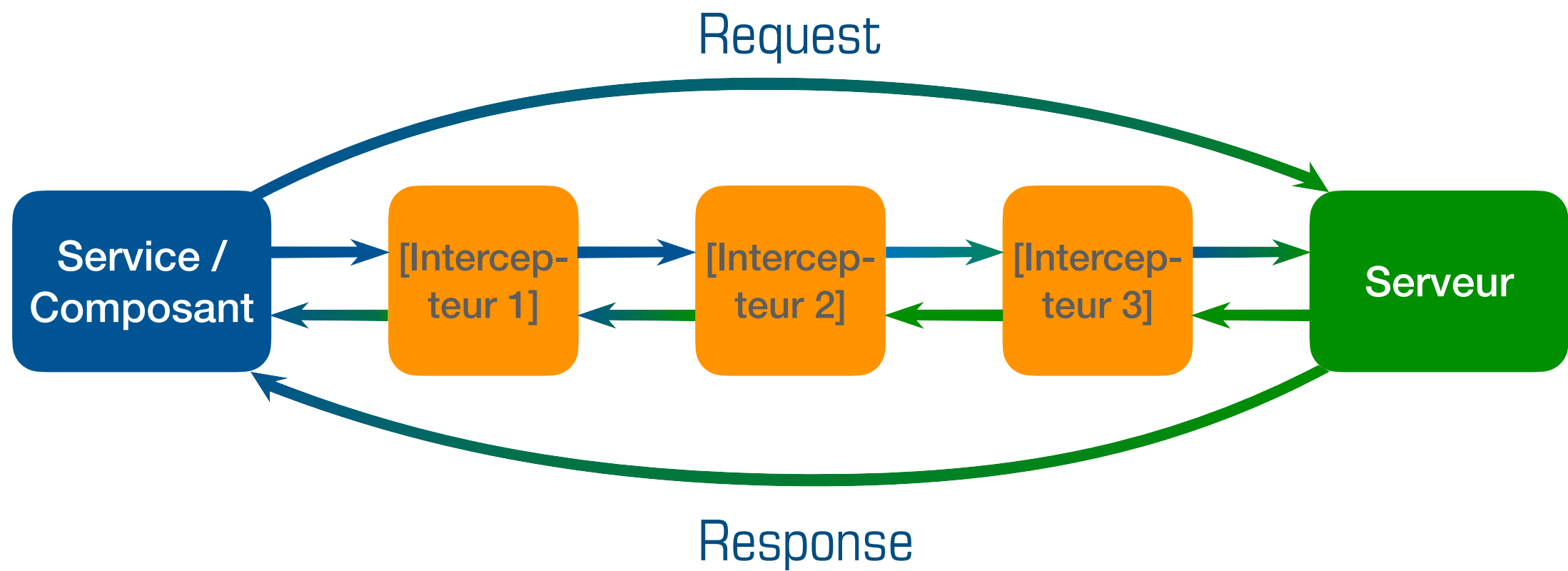
Exemple :

```
.ts
provideHttpClient(
  withInterceptors([
    AuthInterceptor,      // 1. ajoute le token
    LoggingInterceptor    // 2. logge la requête
  ])
)
```

Ici : le **LoggingInterceptor** verra la requête déjà enrichie du token.

Si on inverse : il logge la requête avant ajout du token.

👍 **Bonne pratique** : mettre les intercepteurs “techniques” (auth, headers) avant ceux de “diagnostic” (logs).



10. Exercice 30 : ordre des intercepteurs

👉 **Objectif** : comprendre que l'ordre des intercepteurs modifie le résultat observé.

1. Activez deux intercepteurs dans `app.config.ts` :

a. **AuthInterceptor** : ajoute un header `Authorization: Bearer test-token`.

```
const cloned = req.clone({
  setHeaders: {
    Authorization: `Bearer ${token}`
  }
})
```

b. **LoggingInterceptor** : logge la requête et la réponse.

2. Faites un appel HTTP simple (ex. `GET https://jsonplaceholder.typicode.com/posts`). et observez la console dans DevTools.

3. ...

3. Test 1 : enregistrez les intercepteurs dans cet ordre :
`withInterceptors(
 [AuthInterceptor, LoggingInterceptor])`
Que voyez-vous dans les logs ?
4. Test 2 : inversez l'ordre :
`withInterceptors(
 [LoggingInterceptor, AuthInterceptor])`
Qu'est-ce qui change dans les logs ?
5. Expliquez en 2 phrases ce que vous constatez :
 - a. Quel intercepteur “voit” le header `Authorization` ?
 - b. Dans quel ordre vaut-il mieux enregistrer Auth et Logging en pratique



Récapitulatif – Intercepteurs HTTP

1. Un intercepteur Angular :
 - a. Est un maillon du pipeline `HttpClient`.
 - b. Peut modifier les requêtes sortantes et/ou les réponses entrantes.
 - c. Est déclaré globalement avec `provideHttpClient(withInterceptors([...]))`.
2. Cas d'usage principaux :
 - a. Authentification → ajouter un token (JWT).
 - b. Gestion d'erreurs → comportement uniforme (ex. toast, redirection).
 - c. Observabilité → logs, mesure de durée, debug.
 - d. Ajout d'en-têtes communs → langue, API key, etc.
3. Principes clés :
 - a. Immutabilité → on clone la requête (`req.clone(...)`).
 - b. Chaîne d'intercepteurs → ordre d'appel aller/retour.
 - c. Pas de `subscribe()` → toujours retourner un `Observable`.
 - d. Sobriété → pas de logique métier lourde dans un intercepteur.

12. Compétences acquises

Compétences acquises

Décrire le rôle d'un intercepteur dans Angular.

Expliquer le cycle d'une requête HTTP et la place des intercepteurs.

Implémenter un intercepteur pour enrichir les requêtes (Authorization, headers).

Mettre en œuvre un intercepteur de log (console + durée).

Appliquer une gestion globale des erreurs (401, 500...).

Intégrer un intercepteur d'authentification dans un projet Angular (Festival).

13. Projet Fil Rouge

👉 **Objectif** : Ajouter une couche d'authentification basique dans l'app Festival.

1. Créer un intercepteur `AuthInterceptor`.
2. Ajouter le header `Authorization: Bearer demo-token` à toutes les requêtes `/api/....`
3. Vérifier via l'onglet Network du navigateur que le header est bien présent.

Bonus 🌩 :

Prévoir une exception pour `/api/login` (pas besoin de token).